

Spintly Type 1 Integration Technical Knowledge Base

Purpose

The **Spintly Type 1 Integration Technical Knowledge Base** provides comprehensive documentation for integrating an external application with the Spintly Cloud platform using Type 1 Integration APIs.

This guide consolidates architecture, authentication, setup procedures, API workflows, database synchronization strategies, and best practices into a single reference document. It is intended to help development teams implement, maintain, and troubleshoot integrations while ensuring consistent communication between an organization's backend system and the Spintly platform. This combines the integration overview, setup workflow, and synchronization examples from the original guides into one unified reference.

---

Scope

This document covers the complete lifecycle of a Type 1 Integration, including:

- Integration architecture
- Authentication using OAuth
- Initial configuration
- API usage
- Database mapping
- User lifecycle management
- Permission management
- Validation and testing
- Troubleshooting
- Recommended implementation practices

The document focuses on backend integrations where an organization's application communicates directly with the Spintly Cloud platform through REST APIs.

---

Intended Audience

This guide is designed for technical users involved in implementing or supporting a Type 1 Integration.

| Role                | Purpose                                           |
|---------------------|---------------------------------------------------|
| Backend Developers  | Implement API integration and business logic      |
| API Integrators     | Configure authentication and consume Spintly APIs |
| Solution Architects | Design the integration architecture               |

| Role                        | Purpose                                     |
|-----------------------------|---------------------------------------------|
| QA Engineers                | Validate API workflows and synchronization  |
| Technical Support Engineers | Troubleshoot integration issues             |
| Project Managers            | Understand the overall integration workflow |

---

## Prerequisites

Readers are expected to have:

- Basic understanding of REST APIs
  - Familiarity with HTTP methods and status codes
  - Knowledge of JSON request and response formats
  - Understanding of OAuth authentication
  - Experience with backend application development
  - Access to a Spintly organization and API credentials (for implementation)
- 

## Document Organization

The knowledge base is organized into logical sections that follow the implementation lifecycle.

### Chapter 1 – Introduction

- 1.1 Overview
- 1.2 Purpose
- 1.3 Scope
- 1.4 Intended Audience
- 1.5 Document Organization
- 1.6 What is Type 1 Integration?
- 1.7 Key Benefits
- 1.8 Core Capabilities

### Chapter 2 – Type 1 Integration Architecture

- 2.1 Architecture Overview
- 2.2 System Components
- 2.3 Integration Workflow
- 2.4 Data Flow
- 2.5 Component Responsibilities
- 2.6 Design Principles

### Chapter 3 – Integration Lifecycle

- 3.1 Overview
- 3.2 Environment Setup

- 3.3 Authentication
  - 3.4 Configuration
  - 3.5 Synchronization
  - 3.6 Validation
  - 3.7 Maintenance
- 

## **Part II – Getting Started**

### **Chapter 4 – Environment Setup**

- 4.1 Overview
- 4.2 Prerequisites
- 4.3 Development Environment
- 4.4 Organization Setup
- 4.5 Client Credentials
- 4.6 Postman Configuration
- 4.7 Setup Checklist
- 4.8 Common Setup Issues

### **Chapter 5 – Authentication**

- 5.1 Overview
  - 5.2 OAuth Authentication
  - 5.3 Authentication Workflow
  - 5.4 Client ID & Client Secret
  - 5.5 Access Token Management
  - 5.6 Authorization Header
  - 5.7 Token Lifecycle
  - 5.8 Security Best Practices
  - 5.9 Authentication Checklist
- 

## **Part III – Initial Configuration**

### **Chapter 6 – Initial Configuration**

- 6.1 Overview
  - 6.2 Configuration Requirements
  - 6.3 Sites Configuration
  - 6.4 Roles Configuration
  - 6.5 Access Points Configuration
  - 6.6 Configuration Relationships
  - 6.7 Caching Strategy
  - 6.8 Validation
  - 6.9 Best Practices
- 

## **Part IV – API Reference**

### **Chapter 7 – API Overview**

- 7.1 Overview
- 7.2 API Categories
- 7.3 Request Structure
- 7.4 Response Structure
- 7.5 HTTP Status Codes
- 7.6 API Design Principles
- 7.7 Documentation Template
- 7.8 Implementation Order

## **Chapter 8 – Authentication API**

- 8.1 Overview
- 8.2 Authentication Endpoint
- 8.3 Request Format
- 8.4 Response Format
- 8.5 Error Handling
- 8.6 Best Practices

## **Chapter 9 – Configuration APIs**

- 9.1 Overview
- 9.2 Get Sites API
- 9.3 Get Roles API
- 9.4 Get Access Points API

## **Chapter 10 – User Management APIs**

- 10.1 Overview
- 10.2 User Lifecycle
- 10.3 Create User API
- 10.4 Update User API
- 10.5 Activate User API
- 10.6 Deactivate User API
- 10.7 Delete User API

## **Chapter 11 – Permission Management APIs**

- 11.1 Overview
- 11.2 Assign Permission API
- 11.3 Update Permission API
- 11.4 Retrieve Permissions API
- 11.5 Remove Permission API

## **Chapter 12 – Card Management APIs**

- 12.1 Overview
  - 12.2 Assign Card API
  - 12.3 Retrieve Cards API
  - 12.4 Update Card API
  - 12.5 Revoke Card API
-

## **Part V – Database Integration & Synchronization**

### **Chapter 13 – Database Integration Strategy**

- 13.1 Overview
  - 13.2 Database Mapping
  - 13.3 Mapping Strategy
  - 13.4 User Mapping
  - 13.5 Site Mapping
  - 13.6 Role Mapping
  - 13.7 Recommended Database Schema
  - 13.8 Synchronization Status
  - 13.9 Synchronization Workflow
  - 13.10 Update Synchronization
  - 13.11 Delete Synchronization
  - 13.12 Error Recovery
  - 13.13 Caching Strategy
  - 13.14 Best Practices
  - 13.15 Common Synchronization Issues
- 

## **Part VI – Integration Workflows**

### **Chapter 14 – End-to-End Integration Workflows**

- 14.1 Overview
  - 14.2 Complete Integration Workflow
  - 14.3 Employee Onboarding Workflow
  - 14.4 Employee Update Workflow
  - 14.5 Employee Transfer Workflow
  - 14.6 Employee Suspension Workflow
  - 14.7 Employee Exit Workflow
  - 14.8 Synchronization Failure Workflow
  - 14.9 Recommended Integration Sequence
  - 14.10 Best Practices
  - 14.11 Common Workflow Mistakes
- 

## **Part VII – Validation and Testing**

### **Chapter 15 – Validation, Testing, and Verification**

- 15.1 Overview
- 15.2 Validation Objectives
- 15.3 Integration Validation Workflow
- 15.4 Authentication Validation
- 15.5 Configuration Validation
- 15.6 User Synchronization Validation
- 15.7 Permission Validation
- 15.8 Credential Validation

- 15.9 Smart Access Validation
  - 15.10 Physical Reader Validation
  - 15.11 End-to-End Test Scenario
  - 15.12 Negative Testing
  - 15.13 Performance Testing
  - 15.14 Operational Checklist
  - 15.15 Common Validation Issues
  - 15.16 Best Practices
- 

## **Part VIII – Troubleshooting and Maintenance**

### **Chapter 16 – Troubleshooting and Operational Maintenance**

- 16.1 Overview
  - 16.2 Troubleshooting Process
  - 16.3 Authentication Issues
  - 16.4 Configuration Issues
  - 16.5 User Synchronization Issues
  - 16.6 Permission Issues
  - 16.7 Credential Issues
  - 16.8 API Errors
  - 16.9 Logging Recommendations
  - 16.10 Retry Strategy
  - 16.11 Preventive Maintenance
  - 16.12 Operational Monitoring
  - 16.13 Troubleshooting Checklist
  - 16.14 Best Practices
- 

## **Part IX – Best Practices and Security Guidelines**

### **Chapter 17 – Best Practices and Security Guidelines**

- 17.1 Overview
- 17.2 Authentication Best Practices
- 17.3 API Consumption Best Practices
- 17.4 Database Best Practices
- 17.5 Synchronization Best Practices
- 17.6 Configuration Management
- 17.7 Error Handling Best Practices
- 17.8 Logging Best Practices
- 17.9 Security Best Practices
- 17.10 Performance Best Practices
- 17.11 Operational Best Practices
- 17.12 Scalability Considerations
- 17.13 Production Readiness Checklist
- 17.14 Common Implementation Mistakes
- 17.15 Summary of Recommendations

---

## **Part X – Frequently Asked Questions (FAQ)**

### **Chapter 18 – Frequently Asked Questions**

- 18.1 Overview
  - 18.2 General Questions
  - 18.3 Authentication Questions
  - 18.4 Configuration Questions
  - 18.5 User Management Questions
  - 18.6 Synchronization Questions
  - 18.7 Permission Questions
  - 18.8 Credential Questions
  - 18.9 API Questions
  - 18.10 Database Questions
  - 18.11 Operational Questions
  - 18.12 Security Questions
  - 18.13 Troubleshooting Questions
  - 18.14 Best Practices Questions
  - 18.15 Quick Reference Table
- 

## **Part XI – Glossary & Appendices**

### **Chapter 19 – Glossary and Appendices**

- 19.1 Overview
  - 19.2 Glossary
  - 19.3 Acronyms
  - 19.4 HTTP Status Code Reference
  - 19.5 Recommended API Execution Sequence
  - 19.6 User Lifecycle Summary
  - 19.7 Synchronization Workflow Summary
  - 19.8 Integration Architecture Summary
  - 19.9 Production Readiness Checklist
  - 19.10 References
  - 19.11 Document Revision History
  - 19.12 Suggested Future Enhancements
- 

## **Chapter 1 – Introduction**

### **What is Type 1 Integration?**

Type 1 Integration enables an organization to integrate its own backend application directly with the Spintly Cloud platform through authenticated REST APIs.

Instead of manually performing user management tasks within the Spintly Dashboard, the organization's backend system communicates with Spintly to automate operations such as user

creation, updates, permission assignment, activation, deactivation, and deletion. This approach reduces manual effort and keeps user information synchronized across both systems.

---

### **Why Type 1 Integration?**

Organizations often maintain their own systems—such as HRMS, ERP, Employee Management Systems, or Visitor Management Systems—as the primary source of employee information.

Without integration, administrators must perform the same user management tasks in both the organization's application and the Spintly Dashboard, increasing operational effort and the risk of inconsistencies.

Type 1 Integration addresses this challenge by allowing the organization's backend to automatically synchronize user information with Spintly through API calls whenever changes occur.

---

### **Key Benefits**

Implementing Type 1 Integration provides several advantages:

- Eliminates duplicate manual data entry.
  - Maintains synchronization between the organization's application and Spintly.
  - Reduces administrative effort.
  - Supports automated user lifecycle management.
  - Enables scalable integration for organizations managing a large number of users.
  - Provides a foundation for automated access management workflows.
- 

### **Core Capabilities**

Type 1 Integration supports backend operations such as:

- Authentication using OAuth.
- Retrieving organizational configuration data.
- Managing users.
- Managing permissions.
- Assigning and removing access credentials.
- Synchronizing user information between systems.
- Supporting end-user access through the Spintly Smart Access application.

These capabilities form the basis for implementing secure and automated access management workflows.

## **Chapter 2 – Type 1 Integration Architecture**



---

## 2.1 Introduction

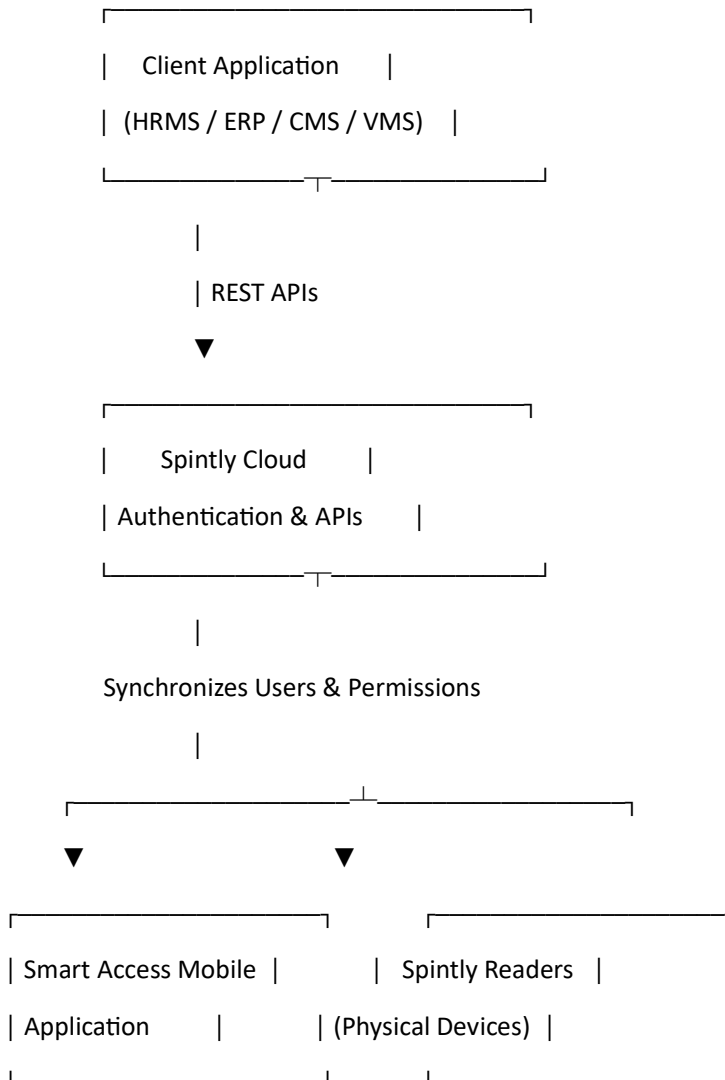
Type 1 Integration establishes secure communication between an organization's backend application and the Spintly Cloud platform. Rather than managing users manually through the Spintly Dashboard, the organization's application becomes the primary system responsible for creating, updating, and maintaining user information.

Whenever user data changes within the organization's application, the backend communicates with Spintly using authenticated REST API requests. This synchronization ensures that both systems remain consistent while reducing manual administration.

---

## 2.2 Architecture Overview

The integration architecture consists of four primary components that work together to provide automated access management.



The Client Application is responsible for business operations, while Spintly Cloud provides authentication, user management, permission management, and access control services. End users interact with the Smart Access application and physical readers to gain authorized access.

---

## 2.3 Core Components

### 2.3.1 Client Application

The Client Application represents the organization's existing software platform. This may be an HRMS, ERP, Employee Management System, Visitor Management System, or any other application responsible for managing users.

Its primary responsibilities include:

- Managing user records.
- Maintaining business logic.
- Initiating synchronization with Spintly.
- Calling authenticated APIs.
- Tracking integration state.
- Managing internal database records.

The Client Application acts as the **System of Record**, meaning user information originates here before being synchronized with Spintly.

---

### 2.3.2 Spintly Cloud

Spintly Cloud is the central platform that exposes REST APIs used by external applications.

It provides services such as:

- OAuth authentication.
- User management.
- Organization management.
- Site management.
- Access point management.
- Permission management.
- Card management.

Every API request from the Client Application is processed by the Spintly Cloud after successful authentication.

---

### 2.3.3 Smart Access Mobile Application

The Smart Access application is installed by end users on their mobile devices.

Its primary functions include:

- User authentication.
- Receiving synchronized user information.
- Managing digital credentials.
- Interacting with authorized readers.
- Providing secure access to assigned locations.

Users only interact with the mobile application after their accounts have been successfully synchronized from the Client Application to Spintly.

---

#### **2.3.4 Spintly Reader**

The Spintly Reader is the physical access control device installed at entry points.

After successful authentication and permission assignment, users can interact with the Reader using the Smart Access application to unlock authorized doors.

The Reader enforces access decisions based on permissions managed through the Spintly platform.

---

### **2.4 Integration Workflow**

The complete integration process follows a predictable sequence.

Administrator

|



Create / Update User

|



Client Application

|



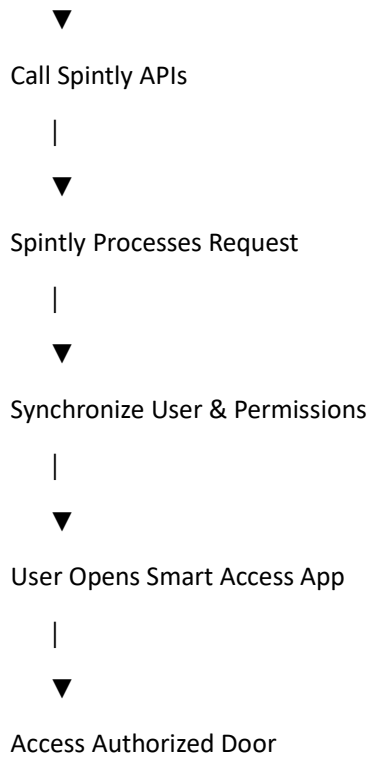
Authenticate with OAuth

|



Receive Access Token

|

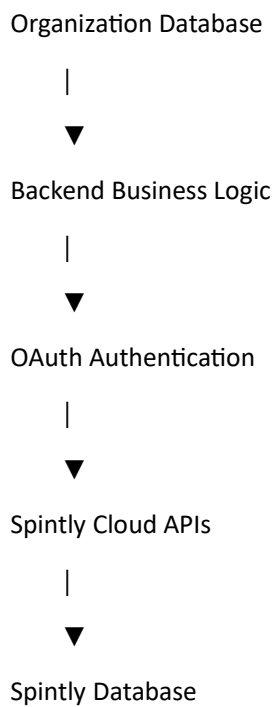


This workflow ensures that the organization's application remains the source of user information while Spintly manages authentication, permissions, and physical access.

---

## 2.5 Data Flow

The movement of information between systems can be summarized as follows:



|



Smart Access Application

|



Physical Reader

Each step depends on the successful completion of the previous one, ensuring secure and reliable synchronization.

---

## 2.6 System Responsibilities

| Component           | Primary Responsibility                             |
|---------------------|----------------------------------------------------|
| Client Application  | Manage business logic and user records             |
| Backend Integration | Authenticate and call Spintly APIs                 |
| Spintly Cloud       | Process requests and manage access control         |
| Smart Access App    | Authenticate users and provide mobile credentials  |
| Spintly Reader      | Grant or deny physical access based on permissions |

---

## 2.7 Design Principles

The integration follows several key principles:

- **Single Source of Truth:** The Client Application owns the master copy of user information.
- **API-Based Synchronization:** All updates are propagated through authenticated REST APIs.
- **Secure Authentication:** Protected APIs require OAuth access tokens.
- **Consistent Data Mapping:** Internal identifiers are mapped to Spintly identifiers to maintain synchronization.
- **Scalable Architecture:** The workflow supports organizations managing large numbers of users without manual intervention.

These principles reduce duplication, improve consistency, and simplify long-term maintenance.

## Chapter 3 – Integration Lifecycle

---

### 3.1 Overview

A Type 1 Integration is not a single API call but a sequence of coordinated steps that enable an organization's backend system to communicate securely with the Spintly Cloud platform.

The integration begins with environment setup and authentication, continues through configuration and user synchronization, and concludes with validation and ongoing maintenance. Understanding this lifecycle helps developers implement the integration correctly and troubleshoot issues more effectively.

---

### **3.2 Integration Phases**

The complete lifecycle can be divided into six phases.

Phase 1

Environment Setup



Phase 2

Authentication



Phase 3

Configuration



Phase 4

User Synchronization



Phase 5

Validation



## Phase 6

### Maintenance

Each phase builds on the previous one and should be completed in order.

---

### **Phase 1 – Environment Setup**

Before any API communication can occur, the integration environment must be prepared.

Typical setup activities include:

- Creating a Spintly organization.
- Accessing the Spintly Dashboard.
- Obtaining Client ID and Client Secret.
- Installing Postman for API testing.
- Importing the provided API collection.

The objective of this phase is to ensure that the development environment is ready for authenticated API communication.

---

### **Phase 2 – Authentication**

Once the environment is prepared, the backend application authenticates with Spintly using OAuth.

The authentication process includes:

1. Sending the Client ID and Client Secret.
2. Receiving an OAuth Access Token.
3. Storing the token securely.
4. Including the token in subsequent API requests.

Without a valid access token, protected APIs cannot be accessed.

---

### **Phase 3 – Configuration**

Before creating users, the backend must retrieve configuration data from Spintly.

This includes:

- Site IDs
- Access Point IDs
- Organization Role IDs

These identifiers are required because user creation and permission assignment depend on existing resources configured within the organization.

---

#### Phase 4 – User Synchronization

After the required configuration data has been collected, the application can synchronize users with Spintly.

The recommended sequence is:

Administrator Creates User



Application Database



Backend Service



Create User API



Spintly Cloud



Return orgUserId



Store orgUserId





## Synchronization Complete

During this phase:

- The application creates the user in its own database.
- The backend invokes the Spintly Create User API.
- Spintly creates the corresponding user.
- The returned spintlyOrgUserId is stored in the application's database for future updates and deletions.

---

## Phase 5 – Validation

After synchronization, the integration should be validated to confirm that the user has been created successfully.

Validation typically includes:

- Verifying the user appears in the Spintly Dashboard.
- Confirming the assigned site.
- Confirming the assigned role.
- Confirming permissions.
- Testing login using the Smart Access application.
- Verifying physical door access through a Spintly Reader.

---

## Phase 6 – Maintenance

After deployment, the integration continues to operate by synchronizing changes made in the organization's application.

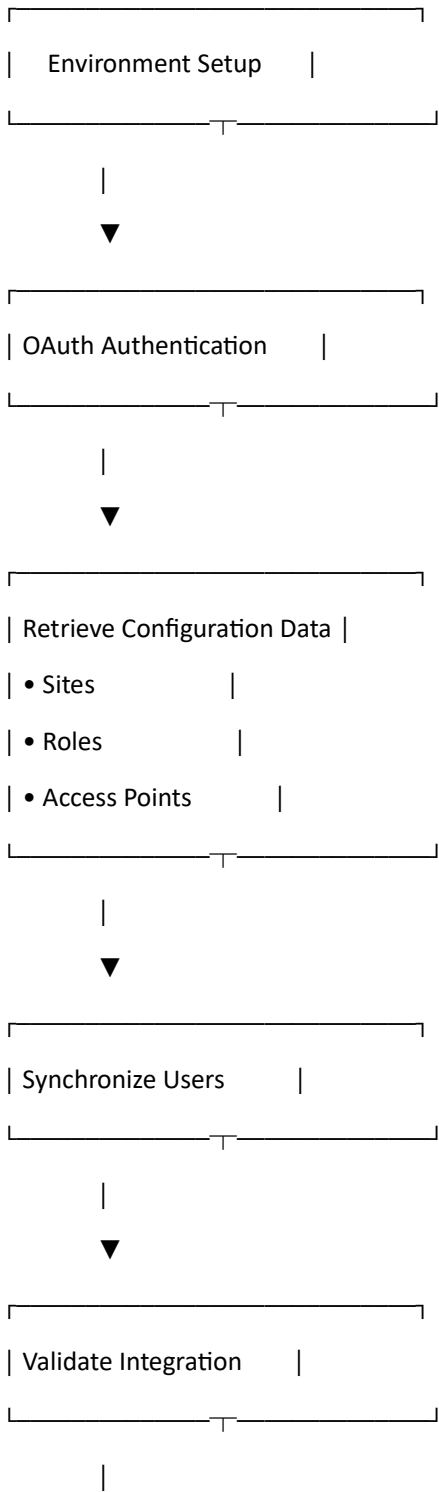
Typical maintenance operations include:

- Updating user information.
- Activating users.
- Deactivating users.
- Deleting users.
- Updating permissions.
- Renewing expired OAuth tokens.
- Monitoring API failures and synchronization logs.

The organization's application remains the primary source of user information throughout the lifecycle.

---

3.3 Complete Lifecycle Diagram





| Maintain & Synchronize |

---

### 3.4 Key Principles

The lifecycle is based on the following principles:

- The organization's backend application is the **single source of truth** for user data.
- All communication with Spintly occurs through authenticated REST APIs.
- Configuration data should be retrieved once and reused where appropriate.
- Spintly identifiers should be stored in the application's database to support future operations.
- Validation should be performed after every significant synchronization step to ensure consistency.

## Chapter 4 – Environment Setup and Prerequisites

---

### 4.1 Overview

Before integrating an application with the Spintly platform, the development environment must be prepared. This includes obtaining the required credentials, configuring the development tools, and verifying access to the Spintly platform.

Completing these prerequisites ensures that the backend application can authenticate successfully and communicate with Spintly APIs during implementation and testing.

---

### 4.2 Prerequisites

The following resources are required before beginning the integration.

| Requirement           | Description                                       |
|-----------------------|---------------------------------------------------|
| Spintly Organization  | A registered organization on the Spintly platform |
| Dashboard Access      | Administrative access to the Spintly Dashboard    |
| Client ID             | OAuth client identifier provided by Spintly       |
| Client Secret         | OAuth client secret provided by Spintly           |
| Internet Connectivity | Required to communicate with Spintly Cloud APIs   |

| Requirement         | Description                                          |
|---------------------|------------------------------------------------------|
| Postman             | Used for testing APIs before application integration |
| Backend Application | The application that will integrate with Spintly     |

---

### 4.3 Create or Access a Spintly Organization

Every Type 1 Integration is associated with a Spintly organization.

If an organization does not already exist:

1. Contact the Spintly team.
2. Request creation of a new organization.
3. Wait for onboarding to be completed.
4. Receive administrator access.

If an organization already exists:

1. Sign in to the Spintly Dashboard.
2. Verify that the organization is accessible.
3. Confirm administrator privileges before continuing.

This organization becomes the environment in which users, sites, permissions, and access points are managed.

---

### 4.4 Dashboard Access

The Spintly Dashboard is the primary administrative interface used during setup and validation.

Developers should verify that they can:

- Log in successfully.
- Access the organization.
- View the User Management section.
- View Sites.
- View Roles.
- View Access Points.

Although user management will later be automated through APIs, dashboard access is essential for validating integration results and troubleshooting issues.

---

### 4.5 Obtain API Credentials

All protected Spintly APIs require OAuth authentication.

To authenticate, developers must obtain:

- Client ID
- Client Secret

These credentials are issued by Spintly during onboarding and uniquely identify the client application.

At the time of writing, these credentials are not generated through a self-service dashboard. They must be requested from Spintly support as part of the onboarding process.

---

#### 4.6 Credential Security

API credentials should be handled securely throughout the development lifecycle.

##### Recommended practices:

- Store credentials in environment variables.
- Never hardcode credentials in source code.
- Do not commit credentials to version control.
- Restrict access to authorized team members.
- Rotate credentials if they are exposed.

Example:

```
CLIENT_ID=xxxxxxxxxxxxxxxxxx
```

```
CLIENT_SECRET=xxxxxxxxxxxxxxxxxx
```

---

#### 4.7 Install Postman

Postman is recommended for validating APIs before integrating them into the backend application.

Using Postman allows developers to:

- Verify authentication.
- Test endpoints independently.
- Validate request payloads.
- Inspect API responses.
- Troubleshoot integration issues before implementation.

Import the Postman collection provided by Spintly and verify that all API requests are available before proceeding.

---

4.8 Development Workflow

The recommended preparation workflow is shown below.

Create / Access Spintly Organization



Verify Dashboard Access



Obtain Client Credentials



Install Postman



Import Spintly Postman Collection



Development Environment Ready

---

4.9 Setup Checklist

Before moving to authentication, verify the following:

| Task                           | Status                   |
|--------------------------------|--------------------------|
| Spintly organization created   | <input type="checkbox"/> |
| Dashboard access verified      | <input type="checkbox"/> |
| Client ID received             | <input type="checkbox"/> |
| Client Secret received         | <input type="checkbox"/> |
| Postman installed              | <input type="checkbox"/> |
| Postman collection imported    | <input type="checkbox"/> |
| Internet connectivity verified | <input type="checkbox"/> |

| Task | Status |
|------|--------|
|------|--------|

|                           |                          |
|---------------------------|--------------------------|
| Backend application ready | <input type="checkbox"/> |
|---------------------------|--------------------------|

Completing this checklist ensures that the environment is prepared for OAuth authentication and subsequent API testing.

---

#### 4.10 Common Setup Issues

| Issue                              | Possible Cause                       | Resolution                                       |
|------------------------------------|--------------------------------------|--------------------------------------------------|
| Unable to log in                   | Incorrect credentials                | Verify dashboard credentials                     |
| Missing Client ID or Client Secret | Onboarding not completed             | Contact Spintly support                          |
| APIs unavailable in Postman        | Collection not imported correctly    | Re-import the latest collection                  |
| Network errors                     | Internet or firewall restrictions    | Verify connectivity and allow required endpoints |
| Unauthorized API requests          | OAuth credentials not yet configured | Complete authentication before testing           |

### Chapter 5 – Authentication

---

#### 5.1 Overview

Spintly secures its REST APIs using the OAuth 2.0 authentication framework. Before a client application can access protected resources, it must authenticate with the Spintly Cloud platform and obtain an access token.

This access token acts as a temporary credential and must be included in the Authorization header of every authenticated API request.

Without a valid access token, requests to protected endpoints will be rejected with an authorization error.

---

#### 5.2 Authentication Workflow

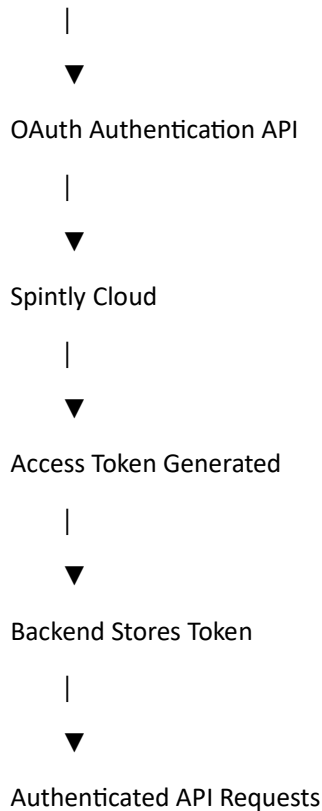
The authentication process consists of the following steps:

Client Application

|



Client ID + Client Secret



The client application authenticates once, receives an access token, and then uses that token for subsequent API calls until it expires.

---

### 5.3 Authentication Components

#### Client ID

The **Client ID** uniquely identifies the integrating application. It is issued during the onboarding process and is required when requesting an OAuth access token.

---

#### Client Secret

The **Client Secret** is a confidential credential associated with the Client ID. It is used to verify the identity of the client application and must be stored securely.

---

#### Access Token

An **Access Token** is returned after successful authentication. It grants temporary authorization to access protected Spintly APIs.

Characteristics:

- Time-limited validity.
- Included in every authenticated request.



- Must be regenerated after expiration.
  - Should never be exposed to end users or stored in insecure locations.
- 

#### 5.4 Authentication Request

The client application sends an authentication request containing its credentials.

Typical authentication parameters include:

| Parameter | Description |
|-----------|-------------|
|-----------|-------------|

|           |                        |
|-----------|------------------------|
| Client ID | Application identifier |
|-----------|------------------------|

|               |                    |
|---------------|--------------------|
| Client Secret | Application secret |
|---------------|--------------------|

|            |                                  |
|------------|----------------------------------|
| Grant Type | OAuth grant type (if applicable) |
|------------|----------------------------------|

The authentication endpoint validates these credentials and returns an access token upon success.

---

#### 5.5 Authentication Response

A successful authentication response typically includes:

| Field | Description |
|-------|-------------|
|-------|-------------|

|              |                                       |
|--------------|---------------------------------------|
| access_token | Token used for authenticated requests |
|--------------|---------------------------------------|

|            |                                          |
|------------|------------------------------------------|
| token_type | Authentication scheme (typically Bearer) |
|------------|------------------------------------------|

|            |                       |
|------------|-----------------------|
| expires_in | Token validity period |
|------------|-----------------------|

Example:

```
{
  "access_token": "eyJhbGciOi...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

**Note:** The exact response fields may vary depending on the OAuth implementation and API version.

---

#### 5.6 Using the Access Token

Once obtained, the access token must be included in the Authorization header of every protected API request.

Example:

Authorization: Bearer <access\_token>

Failure to include the token, or using an expired or invalid token, will result in authentication errors.

---

## 5.7 Token Lifecycle

Generate Token

|



Store Securely

|



Use for API Calls

|



Token Expires

|



Generate New Token

|



Continue API Requests

Applications should monitor token validity and request a new token before or immediately after expiration to avoid interrupted API communication.

---

## 5.8 Security Best Practices

When implementing OAuth authentication:

- Store credentials in environment variables.
- Use HTTPS for all API communication.
- Never expose the Client Secret in frontend applications.
- Do not log access tokens in production environments.
- Limit access to credentials to authorized personnel.
- Rotate credentials if compromise is suspected.

- Validate authentication failures and retry only when appropriate.

These practices help protect sensitive credentials and reduce the risk of unauthorized access.

---

## 5.9 Common Authentication Errors

| HTTP Status                      | Possible Cause                                         | Resolution                                                   |
|----------------------------------|--------------------------------------------------------|--------------------------------------------------------------|
| <b>400 Bad Request</b>           | Missing or invalid request parameters                  | Verify the authentication request payload                    |
| <b>401 Unauthorized</b>          | Invalid Client ID, Client Secret, or expired token     | Verify credentials or request a new access token             |
| <b>403 Forbidden</b>             | Client is authenticated but lacks required permissions | Verify organization permissions and account configuration    |
| <b>500 Internal Server Error</b> | Server-side issue                                      | Retry later or contact Spintly support if the issue persists |

---

## 5.10 Authentication Checklist

Before proceeding to API operations, confirm that:

- Client ID is available.
- Client Secret is available.
- Authentication request succeeds.
- Access token is received.
- Token is stored securely.
- Protected APIs accept the token.
- Token renewal strategy is implemented.

## Chapter 6 – Initial Configuration

---

### 6.1 Overview

After successfully authenticating with the Spintly Cloud platform, the next step is to retrieve the configuration data required for user management and access control.

These configuration resources define the organizational structure within Spintly and provide the identifiers needed for subsequent API operations such as creating users, assigning permissions, and managing access.

The primary configuration resources are:

- Sites

- Roles
- Access Points

These resources are created and managed within the Spintly platform and are referenced by unique identifiers in API requests.

---

## 6.2 Why Initial Configuration is Required

Every user created in Spintly must belong to an existing organizational structure.

For example:

- A user belongs to a **Site**.
- A user is assigned one or more **Roles**.
- Roles determine which **Access Points** (doors/readers) the user can access.

Since these entities already exist in Spintly, the client application must retrieve their identifiers before performing user-related operations.

Without these identifiers, requests such as **Create User** or **Assign Permission** cannot be completed successfully.

---

## 6.3 Configuration Workflow

The recommended sequence for retrieving configuration data is:

Authenticate with OAuth

|



Retrieve Sites

|



Retrieve Roles

|



Retrieve Access Points

|



Store Configuration Data

|



Ready for User Operations

This workflow ensures that all required reference data is available before interacting with user management APIs.

---

## 6.4 Site Configuration

### Purpose

A **Site** represents a physical location or facility managed within the Spintly platform.

Examples include:

- Corporate Office
- Factory
- Warehouse
- Branch Office
- Residential Complex

Every user created in Spintly must be associated with a valid Site.

---

### Retrieve Site Information

The application should call the **Get Sites** API to retrieve the list of available sites.

The response typically contains:

- Site ID
- Site Name
- Site Description
- Status

The **Site ID** returned by the API should be stored by the client application, as it is required when creating or updating users.

---

### Best Practices

- Retrieve sites during initial setup.
  - Refresh site information if organizational changes occur.
  - Store the Site ID rather than relying on the site name.
  - Validate that the selected Site ID exists before using it in API requests.
-

## 6.5 Role Configuration

### Purpose

A **Role** defines the permissions assigned to a user within the organization.

Roles determine the level of access and the areas a user is authorized to enter.

Examples:

- Employee
  - Manager
  - Security Staff
  - Administrator
  - Visitor
- 

### Retrieve Role Information

The client application should use the **Get Roles** API to obtain the available organizational roles.

The response generally includes:

- Role ID
- Role Name
- Description
- Status

The returned **Role ID** should be stored and referenced whenever users are created or their permissions are updated.

---

### Best Practices

- Store Role IDs in the application's database.
  - Avoid using role names as unique identifiers.
  - Synchronize role information periodically if changes are made within Spintly.
  - Validate Role IDs before assigning them to users.
- 

## 6.6 Access Point Configuration

### Purpose

An **Access Point** represents a controlled entry point managed by the Spintly platform.

Examples include:

- Main Entrance
- Server Room
- Parking Gate
- Laboratory
- Conference Room

Access Points are associated with permissions that determine where users can gain physical access.

---

### Retrieve Access Point Information

The application should call the **Get Access Points** API to retrieve the configured access points.

Typical response fields include:

- Access Point ID
- Name
- Location
- Status

The Access Point IDs are later referenced when configuring permissions or assigning access rights.

---

### Best Practices

- Retrieve access points during system initialization.
  - Update cached information when infrastructure changes occur.
  - Validate Access Point IDs before assigning permissions.
  - Avoid hardcoding identifiers in the application.
- 

## 6.7 Configuration Data Relationships

The configuration resources are interconnected.

Organization

|

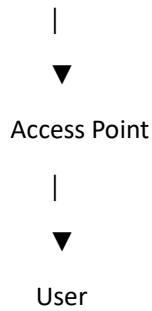


Site

|



Role



Each user is associated with a Site and one or more Roles, while Roles define the Access Points that the user can access.

---

## 6.8 Caching Configuration Data

Since Sites, Roles, and Access Points change infrequently, it is recommended to cache this information within the client application.

Benefits include:

- Reduced API calls.
- Improved application performance.
- Lower network latency.
- Faster user creation and permission assignment.

The cache should be refreshed whenever administrative changes are made within the Spintly platform.

---

## 6.9 Common Configuration Issues

| Issue                 | Possible Cause                  | Resolution                            |
|-----------------------|---------------------------------|---------------------------------------|
| No Sites returned     | Organization not configured     | Verify organization setup             |
| Missing Roles         | Roles not created in Spintly    | Configure roles through the dashboard |
| Invalid Site ID       | Stale cached data               | Refresh configuration                 |
| Invalid Role ID       | Role deleted or modified        | Retrieve latest role information      |
| Missing Access Points | Readers or doors not configured | Verify infrastructure configuration   |

---

## 6.10 Configuration Checklist

Before proceeding to user management, verify that:

- OAuth authentication is successful.



- Site information has been retrieved.
- Role information has been retrieved.
- Access Point information has been retrieved.
- Configuration data has been stored or cached.
- Identifiers have been validated.
- The application is ready to create and manage users.

## Chapter 7 – API Overview

---

### 7.1 Overview

The Spintly Type 1 Integration exposes a collection of REST APIs that enable external applications to interact with the Spintly Cloud platform.

These APIs support the complete lifecycle of user and access management, including authentication, configuration retrieval, user management, permission assignment, and credential management.

All protected APIs require successful OAuth authentication. The access token obtained during authentication must be included in the Authorization header of every request.

---

### 7.2 API Categories

The APIs can be grouped into logical categories based on their purpose.

| Category              | Purpose                                                              |
|-----------------------|----------------------------------------------------------------------|
| Authentication        | Authenticate the client application and obtain an OAuth access token |
| Configuration         | Retrieve Sites, Roles, and Access Points                             |
| User Management       | Create, update, activate, deactivate, and delete users               |
| Permission Management | Assign or update user permissions                                    |
| Card Management       | Manage user access cards and credentials                             |

Grouping APIs by category provides a structured approach to implementation and troubleshooting.

---

### 7.3 API Workflow

A typical integration follows this sequence:

OAuth Authentication

|



Retrieve Sites

|



Retrieve Roles

|



Retrieve Access Points

|



Create User

|



Assign Permissions

|



Manage Cards (if applicable)

|



Update / Deactivate / Delete User

This sequence reflects the recommended implementation order and ensures that all required dependencies are available before performing user operations.

---

## 7.4 Standard API Request Structure

Although each endpoint serves a different purpose, all API requests follow a common structure.

A typical request includes:

- HTTP Method (GET, POST, PUT, DELETE)
- Endpoint URL
- Authorization Header
- Request Headers
- Path Parameters (if applicable)
- Query Parameters (if applicable)
- JSON Request Body (if applicable)

Example:

POST /users

Authorization: Bearer <access\_token>

Content-Type: application/json

Maintaining a consistent request structure simplifies development and debugging.

---

### 7.5 Standard API Response Structure

Responses generally include:

- HTTP Status Code
- Response Body
- Resource Identifier
- Success or Error Message

Example:

```
{
  "success": true,
  "message": "User created successfully.",
  "data": {
    "orgUserId": "xxxxxxx"
  }
}
```

Applications should validate both the HTTP status code and the response body before proceeding with subsequent operations.

---

### 7.6 Common HTTP Status Codes

| Status Code    | Meaning                                                | Typical Scenario          |
|----------------|--------------------------------------------------------|---------------------------|
| 200 OK         | Request processed successfully                         | Data retrieved or updated |
| 201 Created    | Resource created successfully                          | User created              |
| 204 No Content | Operation completed successfully without response body | Delete operation          |

| Status Code                      | Meaning                  | Typical Scenario                |
|----------------------------------|--------------------------|---------------------------------|
| <b>400 Bad Request</b>           | Invalid request          | Missing or incorrect parameters |
| <b>401 Unauthorized</b>          | Authentication failed    | Missing or invalid access token |
| <b>403 Forbidden</b>             | Insufficient permissions | Client lacks authorization      |
| <b>404 Not Found</b>             | Resource does not exist  | Invalid identifier              |
| <b>409 Conflict</b>              | Resource already exists  | Duplicate user                  |
| <b>500 Internal Server Error</b> | Unexpected server error  | Retry or contact support        |

Understanding these status codes helps developers diagnose issues quickly.

---

## 7.7 API Design Principles

The Spintly APIs follow several design principles:

- REST-based communication.
- JSON request and response payloads.
- OAuth-protected endpoints.
- Resource-oriented URLs.
- Standard HTTP methods.
- Consistent response formats.
- Stateless request processing.

These principles make the APIs predictable and easy to integrate.

---

## 7.8 Recommended API Consumption Practices

When integrating with Spintly APIs:

- Authenticate once and reuse the access token until it expires.
- Validate all request payloads before sending them.
- Handle API errors gracefully.
- Store resource identifiers returned by the APIs.
- Avoid unnecessary API calls by caching configuration data.
- Log requests and responses during development for troubleshooting.

---

## 7.9 API Documentation Template

Every API described in the following chapters follows the same structure.

Purpose

Prerequisites

HTTP Method

Endpoint

Headers

Path Parameters

Query Parameters

Request Body

Response

Status Codes

Example Request

Example Response

Common Errors

Best Practices

## Related APIs

Using a consistent format makes the documentation easier to navigate and simplifies implementation.

---

### 7.10 API Implementation Order

For a new integration, implement the APIs in the following order:

#### Step API

- 1 OAuth Authentication
- 2 Get Sites
- 3 Get Roles
- 4 Get Access Points
- 5 Create User
- 6 Update User
- 7 Activate User
- 8 Deactivate User
- 9 Delete User
- 10 Manage Permissions
- 11 Manage Cards

Following this sequence ensures that prerequisites for each operation are satisfied.

## Chapter 8 – Authentication API (OAuth)

---

### 8.1 Overview

The Authentication API is the entry point for all protected interactions with the Spintly Cloud platform. Before an application can retrieve configuration data or manage users, it must first authenticate using its assigned credentials.

Successful authentication returns an OAuth access token that must accompany all subsequent API requests.

---

### 8.2 Purpose

The Authentication API is responsible for:

- Verifying the identity of the client application.

- Issuing a temporary OAuth access token.
- Securing access to protected resources.
- Preventing unauthorized API usage.

Every integration begins with this API.

---

### 8.3 Prerequisites

Before calling this API, ensure that:

- A Spintly organization has been created.
  - Client ID has been issued.
  - Client Secret has been issued.
  - Internet connectivity is available.
  - The authentication endpoint is accessible.
- 

### 8.4 HTTP Method

POST

---

### 8.5 Endpoint

**Note:** Replace the placeholder below with the endpoint provided in the official Postman collection or OpenAPI specification.

POST /oauth/token

*(If the actual endpoint differs, use the official endpoint from Spintly.)*

---

### 8.6 Request Headers

| Header       | Required    | Description                                                                                         |
|--------------|-------------|-----------------------------------------------------------------------------------------------------|
| Content-Type | Yes         | Specifies the request body format (typically application/json or application/x-www-form-urlencoded) |
| Accept       | Recommended | Indicates the expected response format                                                              |

---

### 8.7 Request Parameters

| Parameter     | Required                        | Description                                              |
|---------------|---------------------------------|----------------------------------------------------------|
| client_id     | Yes                             | Unique identifier assigned to the client application     |
| client_secret | Yes                             | Secret credential associated with the client application |
| grant_type    | Depends on OAuth implementation | OAuth grant type required by the authentication server   |

---

## 8.8 Example Request

POST /oauth/token

Content-Type: application/json

```
{
  "client_id": "YOUR_CLIENT_ID",
  "client_secret": "YOUR_CLIENT_SECRET"
}
```

Replace the payload with the format required by the official Spintly authentication endpoint.

---

## 8.9 Successful Response

Example:

```
{
  "access_token": "eyJhbGc...",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

---

## 8.10 Response Fields

| Field        | Description                                     |
|--------------|-------------------------------------------------|
| access_token | OAuth token used for authenticated API requests |
| token_type   | Authentication scheme (typically Bearer)        |



| Field      | Description                        |
|------------|------------------------------------|
| expires_in | Token validity period (in seconds) |

---

### 8.11 Using the Access Token

After authentication, include the access token in every protected request.

Example:

Authorization: Bearer <access\_token>

Example:

GET /sites

Authorization: Bearer eyJhbGc...

---

### 8.12 Authentication Flow

Client Application

|



Client ID + Client Secret

|



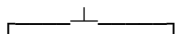
Authentication API

|



Credentials Valid?

|



|

Yes

|

No

|



|



Generate    Return Error

Access Token

|



## Protected API Calls

---

### 8.13 Status Codes

| Status                           | Meaning                               |
|----------------------------------|---------------------------------------|
| <b>200 OK</b>                    | Authentication successful             |
| <b>400 Bad Request</b>           | Missing or invalid request parameters |
| <b>401 Unauthorized</b>          | Invalid credentials                   |
| <b>403 Forbidden</b>             | Client is not authorized              |
| <b>500 Internal Server Error</b> | Unexpected server-side error          |

---

### 8.14 Common Errors

#### Invalid Client ID

##### Cause

The supplied Client ID does not exist.

##### Resolution

Verify the Client ID provided during onboarding.

---

#### Invalid Client Secret

##### Cause

The Client Secret is incorrect.

##### Resolution

Confirm the secret or request a new credential if necessary.

---

#### Expired Token

##### Cause

The access token has exceeded its validity period.

##### Resolution

Generate a new token by calling the Authentication API again.

---

### **Missing Authorization Header**

#### **Cause**

A protected API request was sent without the Authorization header.

#### **Resolution**

Include the header using the Bearer token returned during authentication.

---

### **8.15 Security Recommendations**

To protect authentication credentials:

- Store Client ID and Client Secret in environment variables.
  - Never expose credentials in frontend applications.
  - Do not commit credentials to version control.
  - Use HTTPS for all API communication.
  - Regenerate credentials if they are compromised.
  - Avoid logging access tokens in production environments.
- 

### **8.16 Best Practices**

- Authenticate once and reuse the token until it expires.
  - Monitor token expiration and refresh it proactively.
  - Handle authentication failures gracefully.
  - Validate responses before using the returned token.
  - Store the access token securely in memory or a secure credential store.
- 

### **8.17 Related APIs**

The Authentication API is a prerequisite for all protected APIs, including:

- Get Sites
- Get Roles
- Get Access Points
- Create User
- Update User
- Delete User

- Permission Management
- Card Management

## **Chapter 9 – Configuration APIs**

### **9.1 Overview**

Configuration APIs provide the foundational information required before any user management operations can be performed. These APIs allow the client application to retrieve organizational resources such as Sites, Roles, and Access Points.

The identifiers returned by these APIs are referenced throughout the integration lifecycle and should be stored by the client application for future API requests.

The recommended execution order is:

Authenticate



Retrieve Sites



Retrieve Roles



Retrieve Access Points



Store Configuration IDs



Begin User Management

---

## 9.2 Get Sites API

### Purpose

The **Get Sites API** retrieves all Sites configured within the authenticated Spintly organization.

A Site represents a physical location, building, campus, or facility where users can be assigned.

The Site ID returned by this API is required when creating or updating users.

---

### Prerequisites

- OAuth authentication completed
- Valid Access Token
- Organization configured
- Appropriate permissions

---

### HTTP Method

GET

---

### Endpoint

Replace this placeholder with the official endpoint from the Postman collection.

GET /sites

---

### Headers

| Header        | Required    | Description         |
|---------------|-------------|---------------------|
| Authorization | Yes         | Bearer Access Token |
| Accept        | Recommended | application/json    |

---

### Request Parameters

None

---

### Example Request

GET /sites HTTP/1.1

Authorization: Bearer <access\_token>

---

### Example Response

```
{
  "success": true,
  "data": [
    {
      "siteId": "995wa2",
      "siteName": "Head Office"
    },
    {
      "siteId": "cc421b",
      "siteName": "Warehouse"
    }
  ]
}
```

---

### Response Fields

| Field    | Description            |
|----------|------------------------|
| siteId   | Unique Site identifier |
| siteName | Name of the Site       |

---

### Usage

Store the returned Site ID.

Example:

Head Office

↓

995wa2

When creating a user later:

```
{  
  "siteId":"995wa2"  
}
```

---

## Common Errors

### Status Cause

- |     |                        |
|-----|------------------------|
| 401 | Missing Access Token   |
| 403 | Permission denied      |
| 404 | Organization not found |
| 500 | Server error           |
- 

## Best Practices

- Cache Site IDs.
  - Refresh if administrators add new Sites.
  - Never hardcode Site Names.
  - Always use Site IDs.
- 

## Related APIs

- OAuth
  - Get Roles
  - Get Access Points
  - Create User
- 

## 9.3 Get Roles API

### Purpose

Retrieves all organizational roles available within the authenticated Spintly organization.

Roles determine the permissions and access levels assigned to users.

The Role ID returned by this API is required when creating or updating users.

---

### HTTP Method

GET

---

**Endpoint**

GET /roles

---

**Headers**

Authorization: Bearer Token

---

**Example Response**

```
{
  "data": [
    {
      "roleId": "62mbfh",
      "roleName": "Employee"
    },
    {
      "roleId": "ab72de",
      "roleName": "Visitor"
    }
  ]
}
```

---

**Store**

Employee

↓

62mbfh

Later

```
{
  "roleId": "62mbfh"
```



```
}
```

---

### Best Practices

- Store Role IDs.
  - Refresh when administrators modify roles.
  - Validate Role IDs before creating users.
- 

### Related APIs

- Get Sites
  - Create User
  - Update User
- 

## 9.4 Get Access Points API

### Purpose

Retrieves all configured Access Points associated with the authenticated organization.

Access Points represent physical entry locations such as doors, gates, elevators, or restricted areas.

---

### HTTP Method

GET

---

### Endpoint

GET /access-points

---

### Example Response

```
{
  "data": [
    {
      "accessPointId": "a12345",
      "name": "Main Entrance"
    },
    {
```

```
    "accessPointId":"b67890",  
    "name":"Server Room"  
  }  
]  
}
```

---

## Usage

Access Point IDs are referenced when configuring permissions and access rights.

---

## Best Practices

- Cache Access Point IDs.
  - Refresh when new readers or doors are added.
  - Validate Access Point IDs before assigning permissions.
- 

## Configuration API Workflow

OAuth Authentication

|



Get Sites

|



Store Site IDs

|



Get Roles

|



Store Role IDs

|



Get Access Points

|



Store Access Point IDs

|



Ready for Create User API

## Chapter 10 – User Management APIs

---

### 10.1 Overview

The User Management APIs enable the client application to manage user accounts within the Spintly platform.

These APIs support the complete user lifecycle, including:

- Creating new users
- Updating existing user information
- Activating users
- Deactivating users
- Deleting users

The client application remains the **System of Record**, while Spintly maintains synchronized user information for access control.

---

### 10.2 User Lifecycle

User Created

|



Create User API

|



Store Spintly User ID

|



Update User API (when required)

|



Activate / Deactivate User



Delete User API

One important concept throughout this lifecycle is the **Spintly User ID (spintlyOrgUserId)**. This identifier is returned when a user is created and must be stored by the client application, as it is required for all future operations.

---

### 10.3 Create User API

#### Purpose

The Create User API registers a new user in the authenticated Spintly organization.

This is typically called after a user has been created in the organization's own application.

---

#### Prerequisites

Before calling this API:

- OAuth authentication completed.
- Valid Access Token available.
- Site ID retrieved.
- Role ID retrieved.
- Required configuration data validated.

---

#### HTTP Method

POST

---

#### Endpoint

Replace with the official endpoint from the Postman collection.

POST /users

---

#### Required Information

The request generally includes information such as:

- User name

- Email address
- Mobile number
- Site ID
- Role ID
- Additional organizational attributes (if applicable)

The exact request body should match the official Spintly API specification.

---

#### Example Request

```
{  
  "name": "John Doe",  
  "email": "john@example.com",  
  "mobile": "9876543210",  
  "siteId": "995wa2",  
  "roleId": "62mbfh"  
}
```

---

#### Example Response

```
{  
  "success": true,  
  "data": {  
    "spintlyOrgUserId": "rp41nf"  
  }  
}
```

---

#### Response Fields

| Field            | Description                           |
|------------------|---------------------------------------|
| spintlyOrgUserId | Unique identifier assigned by Spintly |

---

#### Important

Immediately store the returned **spintlyOrgUserId** in your application's database.

Example:

## Application User ID Spintly User ID

|      |        |
|------|--------|
| 1025 | rp41nf |
|------|--------|

Without this mapping, future update or delete operations cannot identify the corresponding user in Spintly.

---

### Best Practices

- Validate all required fields before sending the request.
  - Store the returned spintlyOrgUserId.
  - Prevent duplicate user creation.
  - Log API failures for troubleshooting.
- 

### Common Errors

#### Status Cause

|     |                                 |
|-----|---------------------------------|
| 400 | Invalid request data            |
| 401 | Invalid or expired access token |
| 409 | User already exists             |
| 500 | Server error                    |

---

## 10.4 Update User API

### Purpose

Updates information for an existing user in Spintly.

Typical updates include:

- Name
  - Email
  - Mobile number
  - Site assignment
  - Role assignment
- 

### HTTP Method

PUT

---

**Endpoint**

PUT /users/{spintlyOrgUserId}

---

**Required Information**

- spintlyOrgUserId
  - Updated user details
- 

**Example Request**

```
{  
  "name": "John Smith",  
  "mobile": "9999999999"  
}
```

---

**Best Practices**

- Update only modified fields.
  - Validate user existence before updating.
  - Keep the application database synchronized.
- 

**10.5 Activate User API****Purpose**

Enables a previously inactive user.

Once activated, the user can authenticate and access assigned resources.

---

**HTTP Method**

POST

---

**Endpoint**

POST /users/{spintlyOrgUserId}/activate

---

**Usage**

Typical scenarios:

- Employee rejoins the organization.
  - Temporary suspension ends.
  - User access is restored.
- 

## 10.6 Deactivate User API

### Purpose

Temporarily disables a user without removing their record.

Deactivated users cannot authenticate or access protected resources until reactivated.

---

### HTTP Method

POST

---

### Endpoint

POST /users/{spintlyOrgUserId}/deactivate

---

### Typical Use Cases

- Employee on long leave.
  - Temporary contractor.
  - Security investigation.
  - Account suspension.
- 

## 10.7 Delete User API

### Purpose

Permanently removes a user from the Spintly platform.

Deletion should generally occur only after the user has been removed from the organization's own system and any required data retention policies have been considered.

---

### HTTP Method

DELETE

---



## Endpoint

DELETE /users/{spintlyOrgUserId}

---

## Required Information

- spintlyOrgUserId
- 

## Best Practices

- Confirm the deletion before sending the request.
  - Remove the user mapping from the application database after successful deletion.
  - Log deletion operations for audit purposes.
- 

## Common Errors

### Status Cause

- |     |                                            |
|-----|--------------------------------------------|
| 404 | User not found                             |
| 401 | Authentication failure                     |
| 409 | User cannot be deleted due to dependencies |
| 500 | Server error                               |
- 

## 10.8 Database Synchronization

One of the most critical responsibilities of the client application is maintaining the relationship between its internal users and the corresponding users in Spintly.

Application Database

---

User ID = 1025

↓

Create User API

↓

Spintly



spintlyOrgUserId = rp41nf



Store Mapping



1025 → rp41nf

This mapping ensures that future update, activation, deactivation, and deletion requests target the correct user in Spintly.



### 10.9 Complete User Management Workflow

Create User in Application



Create User API



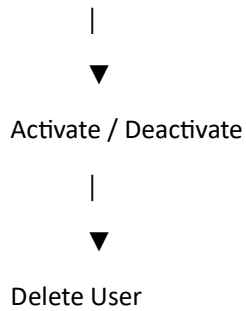
Receive spintlyOrgUserId



Store Mapping



Update User (if required)



---

### 10.10 Best Practices

- Always create the user in the application first.
- Store the returned spintlyOrgUserId immediately.
- Keep application and Spintly records synchronized.
- Validate Site IDs and Role IDs before user creation.
- Handle API failures gracefully with logging and retries where appropriate.
- Maintain audit logs for create, update, activate, deactivate, and delete operations.

## Chapter 11 – Permission Management APIs

---

### 11.1 Overview

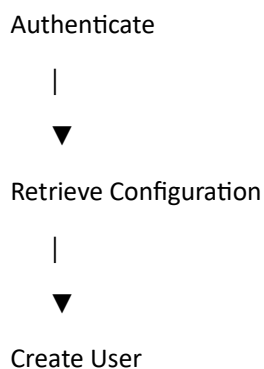
Creating a user in Spintly does not automatically grant access to physical locations or secured resources. Access is controlled through permissions that define where and when a user is authorized to enter.

Permission Management APIs allow the client application to assign, update, retrieve, or revoke permissions associated with users.

These APIs ensure that users receive the correct level of access based on their role and organizational policies.

---

### 11.2 Permission Workflow



|  
▼  
Assign Permission

|  
▼  
Permission Stored

|  
▼  
User Can Access Authorized Locations

Permissions should only be assigned after the user has been successfully created.

---

### 11.3 Permission Components

A permission is typically associated with several organizational resources.

| Component                | Purpose                         |
|--------------------------|---------------------------------|
| User                     | Person receiving access         |
| Site                     | Physical location               |
| Role                     | Defines access level            |
| Access Point             | Door, gate, or controlled entry |
| Schedule (if applicable) | Defines when access is allowed  |

Together, these components determine the user's effective access rights.

---

### 11.4 Assign Permission API

#### Purpose

Assigns one or more permissions to an existing user.

Permissions determine the locations and resources that the user is authorized to access.

---

#### Prerequisites

Before calling this API:

- OAuth authentication completed.
- User already exists.

- spintlyOrgUserId is available.
  - Site and Role information has been retrieved.
  - Access Point configuration is available.
- 

### HTTP Method

POST

---

### Endpoint

Replace with the official endpoint from the Postman collection.

POST /permissions

---

### Example Request

```
{  
  "spintlyOrgUserId": "rp41nf",  
  "roleId": "62mbfh",  
  "siteId": "995wa2"  
}
```

---

### Example Response

```
{  
  "success": true,  
  "message": "Permission assigned successfully."  
}
```

---

### Best Practices

- Validate user existence before assigning permissions.
  - Verify Site ID and Role ID.
  - Prevent duplicate permission assignments.
  - Log permission changes for auditing.
- 

## 11.5 Update Permission API

**Purpose**

Modifies the permissions assigned to an existing user.

Typical changes include:

- Changing roles.
  - Updating accessible locations.
  - Adding or removing access rights.
  - Revising access schedules.
- 

**HTTP Method**

PUT

---

**Endpoint**

PUT /permissions/{permissionId}

---

**Typical Use Cases**

- Employee promotion.
  - Department transfer.
  - Office relocation.
  - Temporary project assignment.
  - Role change.
- 

**11.6 Retrieve User Permissions API****Purpose**

Returns all permissions currently assigned to a user.

This API is commonly used for:

- Validation
  - Troubleshooting
  - Auditing
  - Administrative review
- 

**HTTP Method**

GET

---

#### Endpoint

GET /users/{spintlyOrgUserId}/permissions

---

#### Example Response

```
{
  "permissions": [
    {
      "site": "Head Office",
      "role": "Employee",
      "accessPoint": "Main Entrance"
    }
  ]
}
```

---

### 11.7 Remove Permission API

#### Purpose

Removes one or more permissions from a user.

Removing a permission revokes access to the associated resources without deleting the user account.

---

#### HTTP Method

DELETE

---

#### Endpoint

DELETE /permissions/{permissionId}

---

#### Typical Use Cases

- Employee transfer.
- Role downgrade.

- Temporary access removal.
- End of project.
- Security policy enforcement.

---

## 11.8 Permission Lifecycle

User Created

|



Assign Permission

|



Permission Active

|



Modify Permission

|



Permission Updated

|



Remove Permission

This lifecycle reflects how permissions evolve as user responsibilities change.

---

## 11.9 Common Errors

| Status                  | Cause                           | Resolution                      |
|-------------------------|---------------------------------|---------------------------------|
| <b>400 Bad Request</b>  | Invalid permission data         | Validate request payload        |
| <b>401 Unauthorized</b> | Missing or expired access token | Generate a new OAuth token      |
| <b>403 Forbidden</b>    | Insufficient privileges         | Verify organization permissions |
| <b>404 Not Found</b>    | User or permission not found    | Confirm identifiers             |
| <b>409 Conflict</b>     | Duplicate permission assignment | Check existing permissions      |



| Status                    | Cause             | Resolution               |
|---------------------------|-------------------|--------------------------|
| 500 Internal Server Error | Server-side issue | Retry or contact support |

---

#### 11.10 Best Practices

- Assign permissions only after successful user creation.
  - Always use valid Site IDs and Role IDs.
  - Store permission identifiers if returned by the API.
  - Validate permissions after assignment.
  - Remove obsolete permissions promptly.
  - Maintain audit logs for all permission changes.
  - Avoid assigning duplicate permissions.
- 

#### 11.11 Relationship Between Users and Permissions

Application User

|



Create User API

|



spintlyOrgUserId

|



Assign Permission API

|



Role Assignment

|



Access Point Assignment

|



## User Gains Physical Access

This illustrates how a user progresses from creation to being granted access through assigned permissions.

## Chapter 12 – Card Management APIs

---

### 12.1 Overview

Card Management APIs are responsible for managing the credentials associated with users. A credential represents the identity used by a user to authenticate with the Spintly ecosystem.

Depending on the deployment, credentials may include:

- Mobile credentials (Smart Access App)
- Physical access cards
- Virtual credentials
- Other supported authentication methods

These APIs allow the client application to assign, update, retrieve, and revoke user credentials while maintaining synchronization with the Spintly platform.

---

### 12.2 Card Management Workflow

Create User

|



Assign Permissions

|



Assign Card / Credential

|



Credential Activated

|



User Authenticates

|



Access Granted by Reader

Card assignment is typically performed after the user has been created and the necessary permissions have been assigned.

---

### 12.3 Credential Components

| Component          | Description                                       |
|--------------------|---------------------------------------------------|
| User               | Individual receiving the credential               |
| Credential/Card    | Digital or physical identity assigned to the user |
| Status             | Active, Inactive, Suspended, Revoked              |
| Access Permissions | Permissions associated with the credential        |

---

### 12.4 Assign Card API

#### Purpose

Assigns a credential or access card to an existing user.

The assigned credential enables the user to authenticate using the supported access method.

---

#### Prerequisites

Before calling this API:

- OAuth authentication completed.
  - User exists.
  - spintlyOrgUserId available.
  - User permissions assigned.
  - Credential available for assignment.
- 

#### HTTP Method

POST

---

#### Endpoint

Replace with the official endpoint from the Postman collection.

POST /cards

---

#### Example Request

```
{
  "spintlyOrgUserId": "rp41nf",
  "cardNumber": "CARD-00123456"
}
```

---

#### Example Response

```
{
  "success": true,
  "message": "Card assigned successfully."
}
```

---

### 12.5 Retrieve User Cards API

#### Purpose

Retrieves all credentials currently associated with a user.

This API is useful for:

- Verification
  - Administrative review
  - Troubleshooting
  - Auditing
- 

#### HTTP Method

GET

---

#### Endpoint

GET /users/{spintlyOrgUserId}/cards

---

#### Example Response

```
{
  "cards": [
    {
```

```
    "cardId": "CARD-00123456",  
    "status": "Active"  
  }  
]  
}
```

---

## 12.6 Update Card API

### Purpose

Updates information associated with an existing credential.

Typical updates include:

- Status changes
  - Replacement credentials
  - Metadata updates
- 

### HTTP Method

PUT

---

### Endpoint

PUT /cards/{cardId}

---

## 12.7 Revoke Card API

### Purpose

Revokes a credential assigned to a user.

Revoking a credential immediately prevents it from being used for authentication while leaving the user account intact.

---

### HTTP Method

DELETE

---

### Endpoint

DELETE /cards/{cardId}

---

**Typical Use Cases**

- Lost card
- Stolen card
- Employee resignation
- Card replacement
- Security incident

---

**12.8 Credential Lifecycle**

Assign Credential



Credential Active



Update Credential



Suspend / Revoke



Replace Credential

---

**12.9 Common Errors**

| Status                  | Cause                    | Resolution                  |
|-------------------------|--------------------------|-----------------------------|
| <b>400 Bad Request</b>  | Invalid card information | Validate request payload    |
| <b>401 Unauthorized</b> | Missing or expired token | Generate a new OAuth token  |
| <b>403 Forbidden</b>    | Insufficient permissions | Verify access rights        |
| <b>404 Not Found</b>    | User or card not found   | Confirm identifiers         |
| <b>409 Conflict</b>     | Card already assigned    | Verify existing assignments |

| Status                    | Cause                   | Resolution               |
|---------------------------|-------------------------|--------------------------|
| 500 Internal Server Error | Unexpected server error | Retry or contact support |

---

### 12.10 Best Practices

- Assign credentials only after user creation and permission assignment.
  - Validate credential availability before assignment.
  - Revoke lost or compromised credentials immediately.
  - Maintain audit logs for all credential operations.
  - Periodically review active credentials to ensure they remain valid.
- 

### 12.11 Relationship Between Users, Permissions, and Credentials

Application User

|



Create User

|



Assign Permissions

|



Assign Card / Credential

|



Smart Access App / Physical Card

|



Spintly Reader

|



Access Decision

This illustrates how credentials complete the access control process by linking a user and their permissions to a method of authentication.

## Chapter 13 – Database Integration Strategy

---

### 13.1 Overview

A successful Type 1 Integration requires more than simply invoking REST APIs. The client application must maintain a reliable mapping between its internal records and the corresponding resources in the Spintly platform.

This mapping enables future operations such as updating users, assigning permissions, and deleting users without repeatedly searching for resources.

The application's database therefore acts as the bridge between business data and the Spintly platform.

---

### 13.2 Why Database Mapping is Required

Every organization already maintains its own employee or visitor database.

Example:

Employee Table

Employee ID : 1025

Name : John Doe

Department : IT

When this employee is synchronized with Spintly:

Spintly

spintlyOrgUserId = rp41nf

Both identifiers now refer to the same individual.

Future API calls require the **Spintly identifier**, not the application's internal employee ID.

Without storing this mapping, the application cannot reliably update or delete the correct user.

---

### 13.3 Mapping Strategy

The client application should maintain mappings between internal identifiers and Spintly identifiers.



### Application Spintly

Employee ID spintlyOrgUserId

Location ID spintlySiteId

Role ID spintlyRoleId

These mappings simplify future synchronization and eliminate unnecessary API lookups.

---

### 13.4 User Mapping

Recommended structure:

#### Application User ID Spintly User ID

1025 rp41nf

1026 wd82kl

1027 pq91af

Whenever the **Create User API** succeeds:

1. Receive spintlyOrgUserId
  2. Store it immediately
  3. Use it for all future operations
- 

### 13.5 Site Mapping

Example

#### Application Location Spintly Site

Mumbai Office 995wa2

Pune Office a712hh

Bangalore Office q127ty

Instead of searching Sites every time:

Mumbai Office

↓

995wa2

Use

```
{  
  "siteId":"995wa2"  
}
```

---

### 13.6 Role Mapping

Example

#### Application Role Spintly Role

|          |        |
|----------|--------|
| Employee | 62mbfh |
| Manager  | h781kl |
| Visitor  | bb72fg |

Again,

Store

Employee

↓

62mbfh

instead of repeatedly calling the Roles API.

---

### 13.7 Recommended Database Schema

#### User Table

| Column           | Description                          |
|------------------|--------------------------------------|
| userId           | Internal application user identifier |
| name             | User name                            |
| email            | Email address                        |
| mobile           | Mobile number                        |
| spintlyOrgUserId | User identifier returned by Spintly  |
| spintlySiteId    | Assigned Site                        |

| Column        | Description                               |
|---------------|-------------------------------------------|
| spintlyRoleId | Assigned Role                             |
| syncStatus    | Synchronization status                    |
| lastSyncTime  | Last successful synchronization timestamp |

---

### 13.8 Synchronization Status

Instead of assuming every API call succeeds, maintain synchronization status.

Example:

| Status | Meaning |
|--------|---------|
|--------|---------|

|         |                          |
|---------|--------------------------|
| Pending | Awaiting synchronization |
|---------|--------------------------|

|        |                           |
|--------|---------------------------|
| Synced | Successfully synchronized |
|--------|---------------------------|

|        |                        |
|--------|------------------------|
| Failed | Synchronization failed |
|--------|------------------------|

|       |                   |
|-------|-------------------|
| Retry | Waiting for retry |
|-------|-------------------|

This makes monitoring and troubleshooting much easier.

---

### 13.9 Synchronization Workflow

Application Database

|



New User Created

|

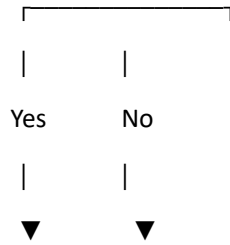


Create User API

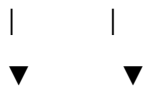
|



Success?



Store      Mark Failed  
spintlyOrgUserId



Update      Retry Later

Database

---

### 13.10 Update Synchronization

When user information changes:

Application



Update Database



Update User API



Success



Update lastSyncTime

The application database should always reflect the latest synchronization state.

---

### **13.11 Delete Synchronization**

Deleting a user should follow this sequence:

Application



Delete User API



Success



Remove Mapping



Archive User

Removing the mapping before confirming API success may result in inconsistent data.

---

### **13.12 Error Recovery**

Synchronization failures are inevitable due to:

- Network interruptions
- Expired OAuth tokens

- Server downtime
- Invalid data
- Timeout errors

Recommended recovery strategy:

1. Record the failure.
2. Preserve the mapping.
3. Retry after resolving the issue.
4. Update the synchronization status upon success.

Avoid deleting or overwriting data until synchronization has completed successfully.

---

### **13.13 Caching Strategy**

Configuration data such as Sites, Roles, and Access Points changes infrequently.

Recommended approach:

- Retrieve during application startup.
- Cache locally.
- Refresh on scheduled intervals or after administrative changes.
- Avoid unnecessary API calls.

Caching improves performance and reduces dependency on repeated network requests.

---

### **13.14 Best Practices**

- Store every Spintly identifier returned by the APIs.
  - Never rely on resource names as unique identifiers.
  - Track synchronization status for each record.
  - Maintain audit logs of synchronization events.
  - Retry failed operations instead of discarding them.
  - Validate mappings before performing update or delete operations.
  - Keep the application database as the source of truth while maintaining consistency with Spintly.
- 

### **13.15 Common Synchronization Issues**

| Problem                | Cause                       | Resolution                                           |
|------------------------|-----------------------------|------------------------------------------------------|
| Update fails           | Missing spintlyOrgUserId    | Store the identifier immediately after user creation |
| Duplicate users        | User created multiple times | Check existing mappings before creating a new user   |
| Invalid Site or Role   | Stale cached identifiers    | Refresh configuration data                           |
| Failed synchronization | Network or server issue     | Retry and update synchronization status              |
| Data inconsistency     | Partial updates             | Use transactional updates and verify API responses   |

## Chapter 14 – End-to-End Integration Workflows

---

### 14.1 Overview

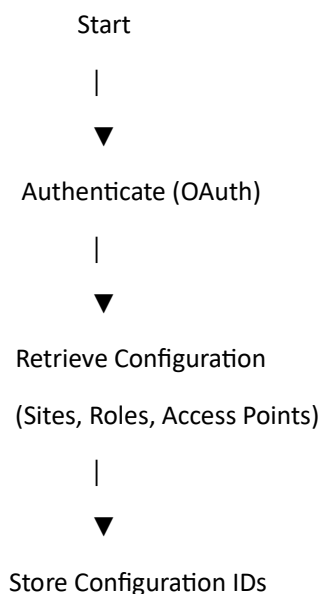
Individual API calls are only one part of a successful integration. In production environments, APIs are executed as part of larger business workflows that synchronize information between the client application and the Spintly platform.

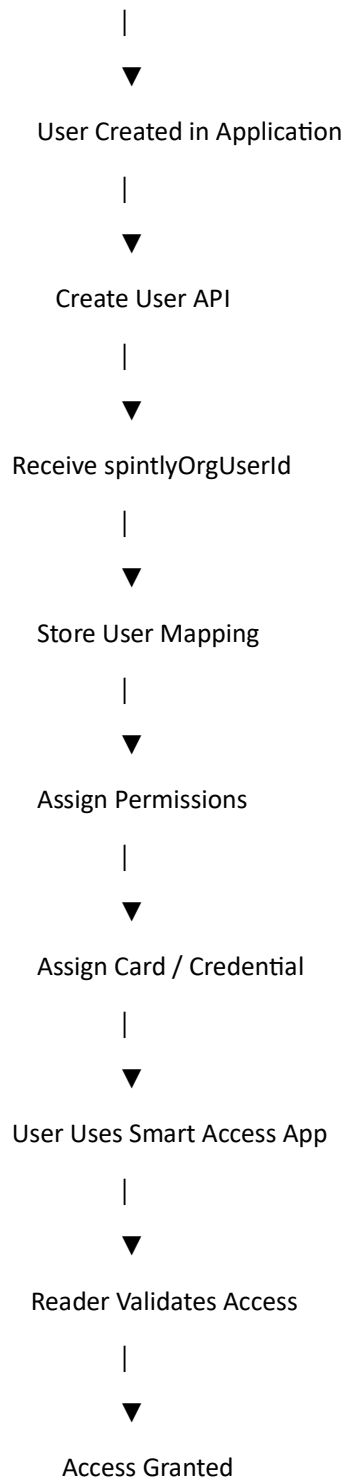
This chapter describes the recommended sequence of operations for common integration scenarios, illustrating how authentication, configuration, user management, permissions, and credential assignment work together.

---

### 14.2 Complete Integration Workflow

The overall Type 1 Integration process is illustrated below.





This workflow represents the complete lifecycle of a newly onboarded user.

---

### 14.3 New Employee Onboarding Workflow

One of the most common use cases is onboarding a new employee.



### **Step 1 – Employee Created**

The HRMS or Employee Management System creates a new employee record.

Employee Created



Employee ID = 1025

---

### **Step 2 – Synchronize with Spintly**

The backend application performs:

- OAuth Authentication
  - Retrieve Site ID
  - Retrieve Role ID
  - Create User API
- 

### **Step 3 – Receive Spintly User ID**

Example:

Application User ID

1025



Create User API



spintlyOrgUserId

rp41nf

Store the mapping immediately.

---

#### **Step 4 – Assign Access**

The application:

- Assigns permissions
- Assigns credential
- Completes provisioning

---

#### **Step 5 – Employee Uses Smart Access**

The employee:

- Installs Smart Access
- Signs in
- Receives credentials
- Uses authorized readers

The onboarding process is now complete.

---

### **14.4 Employee Update Workflow**

Changes to employee information should also be synchronized.

Typical updates include:

- Name change
- Department transfer
- Office relocation
- Mobile number update
- Role change

Workflow:

Application Updated



Update Database



Update User API



Synchronization Successful

---

#### **14.5 Employee Transfer Workflow**

Example:

Employee moves

Mumbai Office



Bangalore Office

Required actions:

- Update Site
  - Update Role (if required)
  - Update Permissions
  - Synchronize database
- 

#### **14.6 Employee Suspension Workflow**

Instead of deleting a user:

Employee Suspended



Deactivate User



Credential Disabled



Access Denied

When reinstated:

Reactivate User



Credential Active



Access Restored

---

#### **14.7 Employee Exit Workflow**

When an employee leaves the organization:

HR Marks Employee Inactive



Deactivate User



Revoke Permissions



Revoke Card



Delete User



Archive Mapping

This sequence ensures access is removed before the user record is permanently deleted.

---

#### **14.8 Synchronization Failure Workflow**

API failures should be handled gracefully.

Application



Create User API



Failure



Log Error



Mark SyncStatus = Failed



Retry



Success



Update Mapping

Never discard synchronization failures without logging or retrying them.

---

#### 14.9 Recommended Integration Sequence

For a new implementation:

##### Phase Action

- 1     Authenticate
- 2     Retrieve configuration
- 3     Cache configuration
- 4     Create user
- 5     Store Spintly identifiers
- 6     Assign permissions
- 7     Assign credentials
- 8     Validate access
- 9     Monitor synchronization

Following this sequence minimizes implementation errors.

---

#### 14.10 Best Practices

- Treat the application database as the source of truth.
- Synchronize changes immediately after business events.
- Store all Spintly identifiers.

- Retry transient failures.
- Log every synchronization event.
- Audit user lifecycle operations.
- Validate configuration before provisioning users.

---

#### 14.11 Common Workflow Mistakes

| Mistake                                    | Impact              | Recommendation                   |
|--------------------------------------------|---------------------|----------------------------------|
| Creating users before retrieving Site IDs  | User creation fails | Retrieve configuration first     |
| Not storing spintlyOrgUserId               | Future updates fail | Store immediately after creation |
| Assigning permissions before user creation | API error           | Create user first                |
| Deleting users before revoking access      | Inconsistent state  | Revoke access before deletion    |
| Ignoring failed API calls                  | Data inconsistency  | Log, retry, and monitor failures |

### Chapter 15 – Validation, Testing, and Verification

---

#### 15.1 Overview

After implementing the integration, it is essential to verify that all components are functioning correctly. Validation ensures that the client application, Spintly Cloud platform, Smart Access application, and physical readers work together as expected.

Testing should be performed during initial implementation, after configuration changes, and following application updates.

---

#### 15.2 Validation Objectives

The primary objectives of validation are to:

- Verify successful authentication.
  - Confirm API connectivity.
  - Validate configuration data.
  - Ensure users are synchronized correctly.
  - Verify permissions and credentials.
  - Confirm physical access through Spintly Readers.
  - Detect synchronization issues before production deployment.
-

### 15.3 Integration Validation Workflow

Authenticate

|



Configuration APIs

|



Create Test User

|



Assign Permission

|



Assign Credential

|



Login to Smart Access

|



Access Physical Reader

|



Integration Successful

---

### 15.4 Authentication Validation

Verify that:

- OAuth authentication succeeds.
- Access token is returned.
- Token is accepted by protected APIs.
- Expired tokens are handled correctly.

**Expected Result**



- HTTP 200 response
  - Valid access token
  - Successful API authentication
- 

### **15.5 Configuration Validation**

Confirm that the application successfully retrieves:

- Sites
- Roles
- Access Points

Verify that:

- Returned identifiers match the Spintly Dashboard.
  - Cached configuration is up to date.
  - Invalid identifiers are rejected.
- 

### **15.6 User Synchronization Validation**

Create a test user in the client application.

Verify that:

- User is created in Spintly.
  - spintlyOrgUserId is returned.
  - Mapping is stored in the application database.
  - Synchronization status is marked as successful.
- 

### **15.7 Permission Validation**

Confirm that:

- Permissions are assigned successfully.
  - Assigned roles match expectations.
  - User has access only to authorized locations.
  - Permission updates are reflected correctly.
- 

### **15.8 Credential Validation**

Verify that:

- Credentials are assigned successfully.
  - Smart Access application displays the credential.
  - Revoked credentials are no longer usable.
  - Reassigned credentials function correctly.
- 

### 15.9 Smart Access Validation

Install the Smart Access application and verify that the user can:

- Sign in successfully.
- View assigned credentials.
- Synchronize account information.
- Access authorized locations.

The application should display only the permissions associated with the authenticated user.

---

### 15.10 Physical Reader Validation

Perform physical access testing.

Verify that:

- Authorized users gain access.
- Unauthorized users are denied.
- Access events are recorded.
- Reader communication is functioning correctly.

Example validation scenarios:

| Scenario              | Expected Result |
|-----------------------|-----------------|
| Authorized employee   | Access granted  |
| Unauthorized employee | Access denied   |
| Deactivated user      | Access denied   |
| Revoked credential    | Access denied   |

---

### 15.11 End-to-End Test Scenario

Execute the following sequence:

Create Employee



Synchronize with Spintly



Store Mapping



Assign Permission



Assign Credential



Login to Smart Access



Open Authorized Door



Success

This validates the complete integration lifecycle.

---

## 15.12 Negative Testing

Test common failure scenarios:

| Scenario                     | Expected Outcome             |
|------------------------------|------------------------------|
| Invalid Client Secret        | Authentication fails         |
| Expired Access Token         | API returns 401 Unauthorized |
| Invalid Site ID              | User creation fails          |
| Invalid Role ID              | Permission assignment fails  |
| Duplicate User               | API returns conflict         |
| Missing Authorization Header | Request rejected             |

Negative testing ensures that the application handles errors gracefully.

---

### 15.13 Performance Testing

Evaluate:

- Authentication response time.
- API response times.
- Bulk user synchronization performance.
- Retry behavior during network interruptions.
- Cache effectiveness for configuration data.

Record metrics and monitor for performance degradation over time.

---

### 15.14 Operational Checklist

Before deploying to production, verify that:

| Task                           | Status                   |
|--------------------------------|--------------------------|
| OAuth authentication verified  | <input type="checkbox"/> |
| Configuration APIs tested      | <input type="checkbox"/> |
| User creation validated        | <input type="checkbox"/> |
| User updates validated         | <input type="checkbox"/> |
| Permission assignment verified | <input type="checkbox"/> |
| Credential assignment verified | <input type="checkbox"/> |

| Task                          | Status                   |
|-------------------------------|--------------------------|
| Smart Access tested           | <input type="checkbox"/> |
| Reader access verified        | <input type="checkbox"/> |
| Error handling validated      | <input type="checkbox"/> |
| Synchronization logs reviewed | <input type="checkbox"/> |

---

### 15.15 Common Validation Issues

| Issue                             | Possible Cause                     | Resolution                            |
|-----------------------------------|------------------------------------|---------------------------------------|
| User not visible in Spintly       | Synchronization failed             | Review API logs and retry             |
| Smart Access shows no credentials | Credential assignment incomplete   | Verify card/credential API            |
| Reader denies authorized user     | Permission configuration incorrect | Check roles and access points         |
| Repeated synchronization failures | Network or authentication issues   | Validate OAuth token and connectivity |
| Duplicate users                   | Missing identifier mapping         | Verify spintlyOrgUserId storage       |

---

### 15.16 Best Practices

- Validate every major workflow before production deployment.
- Use dedicated test accounts.
- Maintain audit logs for all validation activities.
- Automate integration tests where possible.
- Repeat validation after configuration changes or software updates.
- Regularly review synchronization logs and API responses.

## Chapter 16 – Troubleshooting and Operational Maintenance

---

### 16.1 Overview

Even with a correctly implemented integration, operational issues such as authentication failures, network interruptions, invalid configuration data, or synchronization errors may occur.

This chapter provides guidance for identifying, diagnosing, and resolving common issues encountered during the lifecycle of a Type 1 Integration. It also outlines recommended maintenance activities to ensure long-term reliability and performance.

---

### 16.2 Troubleshooting Process

When an issue occurs, follow a structured troubleshooting approach.

Issue Detected

|



Collect Error Details

|



Review Application Logs

|



Verify OAuth Authentication

|



Validate API Request

|



Check Response Code

|



Resolve Issue

|



Retry Operation

This process helps isolate the root cause before applying corrective actions.

---

### 16.3 Authentication Issues

#### Symptoms

- API returns **401 Unauthorized**
- Unable to generate an access token
- Authentication fails intermittently

#### **Possible Causes**

- Invalid Client ID
- Invalid Client Secret
- Expired OAuth token
- Incorrect Authorization header

#### **Resolution**

- Verify Client ID and Client Secret.
  - Generate a new OAuth access token.
  - Ensure the Authorization: Bearer <access\_token> header is included.
  - Confirm system time is synchronized if token expiry is time-based.
- 

### **16.4 Configuration Issues**

#### **Symptoms**

- Invalid Site ID
- Invalid Role ID
- Missing Access Point

#### **Possible Causes**

- Cached configuration is outdated.
- Configuration was modified in the Spintly Dashboard.
- Incorrect identifier used in API requests.

#### **Resolution**

- Refresh configuration using the Configuration APIs.
  - Update the local cache.
  - Validate identifiers before sending requests.
- 

### **16.5 User Synchronization Issues**

#### **Symptoms**

- User exists in the application but not in Spintly.

- Update operations fail.
- Delete requests return "User Not Found."

#### **Possible Causes**

- Create User API failed.
- spintlyOrgUserId not stored.
- Database mapping is missing.

#### **Resolution**

- Verify the Create User API response.
  - Store the returned spintlyOrgUserId.
  - Rebuild mappings if necessary.
- 

### **16.6 Permission Issues**

#### **Symptoms**

- User exists but cannot access authorized locations.
- Permission assignment fails.
- Reader denies access.

#### **Possible Causes**

- Incorrect Role ID.
- Permission not assigned.
- Invalid Access Point.

#### **Resolution**

- Verify role assignments.
  - Reassign permissions.
  - Confirm Access Point configuration.
- 

### **16.7 Credential Issues**

#### **Symptoms**

- Smart Access application shows no credential.
- Physical card is rejected.
- Credential appears inactive.

#### **Possible Causes**



- Credential not assigned.
- Credential revoked.
- Synchronization incomplete.

#### Resolution

- Verify card assignment.
- Reassign or replace the credential.
- Confirm synchronization completed successfully.

---

### 16.8 API Errors

| HTTP Status Meaning |                       | Recommended Action            |
|---------------------|-----------------------|-------------------------------|
| 400                 | Invalid request       | Validate request payload      |
| 401                 | Unauthorized          | Generate a new OAuth token    |
| 403                 | Forbidden             | Verify permissions            |
| 404                 | Resource not found    | Confirm identifiers           |
| 409                 | Conflict              | Check for duplicate resources |
| 500                 | Internal server error | Retry and review logs         |

---

### 16.9 Logging Recommendations

Maintain application logs for:

- Authentication requests
- API responses
- Synchronization events
- Permission changes
- Credential operations
- Error messages
- Retry attempts

Logs should include timestamps, request identifiers, and relevant response codes while avoiding sensitive information such as access tokens or client secrets.

---

### 16.10 Retry Strategy

Some failures are temporary and should be retried.

Examples include:

- Temporary network interruptions
- Service timeouts
- HTTP 500 responses

Avoid retrying requests caused by invalid credentials or malformed payloads until the underlying issue has been corrected.

---

### 16.11 Preventive Maintenance

Regular maintenance helps ensure reliable operation.

Recommended activities:

- Refresh cached configuration data periodically.
- Monitor synchronization status.
- Review application logs.
- Remove obsolete mappings.
- Test OAuth authentication regularly.
- Validate critical workflows after configuration changes.
- Keep API documentation and integration code aligned with the latest Spintly API version.

---

### 16.12 Operational Monitoring

Monitor the following metrics:

| Metric                         | Purpose                             |
|--------------------------------|-------------------------------------|
| Authentication success rate    | Detect login issues                 |
| API response time              | Measure performance                 |
| Synchronization success rate   | Verify data consistency             |
| Failed API requests            | Identify recurring issues           |
| Credential assignment failures | Detect provisioning problems        |
| Reader access failures         | Monitor physical access reliability |

Monitoring these metrics helps identify problems before they affect users.

---

### 16.13 Troubleshooting Checklist

Before escalating an issue, verify:

- OAuth authentication is valid.
  - Access token has not expired.
  - Site IDs and Role IDs are current.
  - User exists in both systems.
  - spintlyOrgUserId is stored correctly.
  - Permissions are assigned.
  - Credentials are active.
  - API logs have been reviewed.
- 

### 16.14 Best Practices

- Log every integration event.
- Monitor synchronization continuously.
- Cache configuration responsibly and refresh it when needed.
- Use retries only for transient failures.
- Keep database mappings consistent.
- Perform regular health checks on the integration.
- Maintain versioned API documentation.

## Chapter 17 – Best Practices and Security Guidelines

---

### 17.1 Overview

Implementing a successful Type 1 Integration involves more than simply consuming REST APIs. A production-ready integration should be secure, reliable, maintainable, and scalable.

This chapter consolidates recommended practices for authentication, API usage, synchronization, security, monitoring, and operational maintenance. Following these recommendations helps minimize implementation errors, improve system reliability, and simplify long-term maintenance.

---

### 17.2 Authentication Best Practices

Authentication is the foundation of every integration.

Recommendations:

- Store **Client ID** and **Client Secret** in environment variables.

- Never hardcode credentials in source code.
  - Always use HTTPS for API communication.
  - Generate OAuth tokens only when required.
  - Reuse valid access tokens until they expire.
  - Refresh expired tokens before retrying protected API requests.
  - Never expose access tokens or client secrets in logs or frontend applications.
- 

### 17.3 API Consumption Best Practices

When interacting with Spintly APIs:

- Validate request payloads before sending them.
- Check both the HTTP status code and response body.
- Handle API failures gracefully.
- Use meaningful error messages in the client application.
- Avoid unnecessary API calls.
- Keep API implementations modular and reusable.
- Follow the recommended API execution order.

Recommended order:

Authenticate



Configuration APIs



Create User



Assign Permissions



Assign Credential



Update / Delete

---

#### 17.4 Database Best Practices

The application database should remain the **System of Record**.

Recommendations:

- Store spintlyOrgUserId immediately after user creation.
- Store Site IDs and Role IDs.
- Maintain synchronization status.
- Keep identifier mappings consistent.
- Use database transactions where appropriate.
- Preserve audit history for critical operations.

Example mapping:

##### Application ID Spintly ID

|      |        |
|------|--------|
| 1025 | rp41nf |
|------|--------|

---

#### 17.5 Synchronization Best Practices

Maintain consistency between the application and Spintly.

Recommendations:

- Synchronize immediately after business events.
  - Retry transient failures.
  - Track synchronization timestamps.
  - Avoid duplicate synchronization.
  - Log synchronization results.
  - Validate synchronization after completion.
-

17.6 Configuration Management

Configuration data changes infrequently but remains essential.

Recommended approach:

- Retrieve Sites, Roles, and Access Points during initialization.
- Cache configuration locally.
- Refresh the cache after administrative changes.
- Avoid hardcoding identifiers.

---

17.7 Error Handling Best Practices

Applications should anticipate and recover from failures.

Recommended approach:

API Call

↓

Success?

|

|— Yes

| Continue

|

|— No

↓

Log Error



Determine Cause



Retry (if applicable)



Notify Administrator

Do not silently ignore failed API requests.

---

### **17.8 Logging Best Practices**

Maintain structured logs for:

- Authentication events
- API requests
- API responses
- Synchronization events
- Permission changes
- Credential operations
- Retry attempts
- Unexpected exceptions

Avoid logging:

- Access tokens
  - Client secrets
  - Sensitive personal information
- 

### **17.9 Security Best Practices**

Security should be incorporated into every stage of the integration.

Recommendations:

- Use HTTPS exclusively.
  - Protect API credentials.
  - Apply the principle of least privilege.
  - Validate all input data.
  - Sanitize user-provided values.
  - Restrict administrative access.
  - Rotate credentials periodically.
  - Monitor authentication failures.
- 

### **17.10 Performance Best Practices**

Improve efficiency by:

- Caching configuration data.
  - Reusing OAuth tokens.
  - Reducing redundant API calls.
  - Using efficient database indexing for identifier mappings.
  - Monitoring API response times.
  - Processing large synchronization jobs in batches where supported.
- 

### **17.11 Operational Best Practices**

During production operation:

- Monitor synchronization health.
  - Review API failures regularly.
  - Test authentication periodically.
  - Validate configuration after organizational changes.
  - Monitor physical reader activity.
  - Review audit logs.
  - Update integration documentation alongside application changes.
- 

### **17.12 Scalability Considerations**

As organizations grow:

- Design for increasing user volumes.



- Maintain efficient identifier lookups.
- Optimize database queries.
- Separate configuration retrieval from operational workflows.
- Consider asynchronous processing for bulk synchronization tasks.
- Monitor infrastructure capacity and API throughput.

---

### 17.13 Production Readiness Checklist

Before deploying to production:

| Item                            | Status                   |
|---------------------------------|--------------------------|
| OAuth implemented               | <input type="checkbox"/> |
| Credentials stored securely     | <input type="checkbox"/> |
| Configuration cached            | <input type="checkbox"/> |
| User mapping implemented        | <input type="checkbox"/> |
| Synchronization tested          | <input type="checkbox"/> |
| Permission assignment validated | <input type="checkbox"/> |
| Credential assignment verified  | <input type="checkbox"/> |
| Logging enabled                 | <input type="checkbox"/> |
| Monitoring configured           | <input type="checkbox"/> |
| Error handling implemented      | <input type="checkbox"/> |
| Retry strategy implemented      | <input type="checkbox"/> |
| Documentation updated           | <input type="checkbox"/> |

---

### 17.14 Common Implementation Mistakes

| Mistake                       | Impact                 | Recommendation                     |
|-------------------------------|------------------------|------------------------------------|
| Hardcoding credentials        | Security risk          | Use environment variables          |
| Not storing spintlyOrgUserId  | Update/Delete failures | Persist the identifier immediately |
| Ignoring failed API responses | Data inconsistency     | Validate and log every response    |

| Mistake                           | Impact                    | Recommendation                 |
|-----------------------------------|---------------------------|--------------------------------|
| Using names instead of IDs        | Invalid references        | Always use Spintly identifiers |
| Excessive configuration API calls | Performance degradation   | Cache configuration data       |
| Missing synchronization logs      | Difficult troubleshooting | Maintain structured logs       |

---

## 17.15 Summary of Recommendations

The following principles should guide every Type 1 Integration:

- Keep the application database as the source of truth.
- Use OAuth securely.
- Store and maintain identifier mappings.
- Validate all API interactions.
- Cache configuration data.
- Monitor synchronization continuously.
- Handle failures gracefully.
- Maintain detailed operational logs.
- Review and update the integration regularly.

## Chapter 18 – Frequently Asked Questions

---

### 18.1 Overview

This chapter answers frequently asked questions related to Type 1 Integration, API usage, authentication, synchronization, and troubleshooting. It serves as a quick reference for developers, system administrators, QA engineers, and support teams during implementation and maintenance.

---

### 18.2 General Questions

#### Q1. What is Type 1 Integration?

**Answer:**

Type 1 Integration allows a client application to integrate directly with the Spintly Cloud Platform using REST APIs. The client application manages users, permissions, and credentials while Spintly provides authentication, access management, and communication with Smart Access applications and physical readers.

---

#### Q2. Who should use Type 1 Integration?

**Answer:**

Type 1 Integration is intended for organizations that already maintain their own application or database and want to synchronize users, permissions, and access credentials with the Spintly platform.

---

**Q3. What are the prerequisites?****Answer:**

Before implementation, ensure that you have:

- A registered organization in Spintly.
  - Client ID and Client Secret.
  - Access to the Spintly Dashboard.
  - API documentation.
  - Development environment with internet connectivity.
- 

**18.3 Authentication Questions****Q4. Why is OAuth required?****Answer:**

OAuth provides secure authentication between the client application and the Spintly platform. A valid access token is required to invoke protected APIs.

---

**Q5. When should I generate an access token?****Answer:**

Generate an access token before making protected API requests and reuse it until it expires.

---

**Q6. What should I do if I receive a 401 Unauthorized response?****Answer:**

Verify:

- Client ID
- Client Secret
- Access Token
- Authorization header

Generate a new access token if the current one has expired.

---

## 18.4 Configuration Questions

### Q7. Why should I retrieve Sites and Roles before creating users?

**Answer:**

User creation requires valid Site IDs and Role IDs. These identifiers should be retrieved and stored before creating users.

---

### Q8. Should I call the Configuration APIs every time?

**Answer:**

No. Configuration data changes infrequently. Retrieve it during initialization, cache it locally, and refresh it only when necessary.

---

## 18.5 User Management Questions

### Q9. Why must I store spintlyOrgUserId?

**Answer:**

This identifier uniquely represents the user within the Spintly platform and is required for future update, permission, credential, and delete operations.

---

### Q10. Can I use employee names instead of IDs?

**Answer:**

No. Always use the identifiers returned by Spintly APIs rather than names, which may not be unique.

---

### Q11. Can I delete a user immediately?

**Answer:**

It is recommended to revoke permissions and credentials before deleting the user to avoid orphaned access records.

---

## 18.6 Synchronization Questions

### Q12. What happens if synchronization fails?

**Answer:**

Record the failure, preserve existing mappings, resolve the issue, and retry the synchronization. Do not delete or overwrite data prematurely.

---

**Q13. How do I avoid duplicate users?****Answer:**

Check whether a mapping already exists before creating a new user. Store the returned spintlyOrgUserId immediately after successful creation.

---

**18.7 Permission Questions****Q14. Why is access denied even though the user exists?****Answer:**

Possible reasons include:

- Permission not assigned.
  - Incorrect Role ID.
  - Invalid Access Point.
  - Credential not assigned or inactive.
- 

**Q15. Can one user have multiple permissions?****Answer:**

Yes, provided the integration and platform configuration support multiple permission assignments.

---

**18.8 Credential Questions****Q16. Why doesn't the Smart Access application display any credentials?****Answer:**

Possible causes include:

- Credential assignment incomplete.
  - Synchronization pending.
  - User not authenticated correctly.
  - Credential revoked.
- 

**Q17. What happens when a credential is revoked?****Answer:**

The user can no longer use that credential for authentication until a valid credential is assigned again.

---

## 18.9 API Questions

### Q18. Which HTTP methods are commonly used?

**Answer:**

Most integrations use:

- GET – Retrieve information
- POST – Create resources
- PUT/PATCH – Update resources
- DELETE – Remove resources

The exact method depends on the API endpoint.

---

### Q19. What should I do if an API returns HTTP 500?

**Answer:**

Review the request, examine application logs, and retry the operation if the error appears to be temporary. Persistent server errors should be reported to the service administrator.

---

## 18.10 Database Questions

### Q20. Which identifiers should I store?

**Answer:**

At a minimum, store:

- spintlyOrgUserId
- spintlySiteId
- spintlyRoleId

These identifiers are required for future synchronization and API operations.

---

### Q21. Should my application or Spintly be the source of truth?

**Answer:**

The client application should remain the primary source of truth, while Spintly should reflect synchronized data for access management.

---

## 18.11 Operational Questions

### Q22. What should I monitor in production?

**Answer:**

Monitor:

- Authentication success rate.
  - API response times.
  - Synchronization failures.
  - Permission assignment failures.
  - Credential assignment status.
  - Reader access events.
  - Application logs.
- 

**Q23. How often should configuration data be refreshed?**

**Answer:**

Refresh configuration whenever administrative changes occur or according to the organization's maintenance schedule.

---

## **18.12 Security Questions**

**Q24. Where should Client Secret be stored?**

**Answer:**

Store it securely in environment variables or a dedicated secrets management solution. Never expose it in source code or frontend applications.

---

**Q25. Can access tokens be logged?**

**Answer:**

No. Access tokens are sensitive credentials and should never be recorded in application logs.

---

## **18.13 Troubleshooting Questions**

**Q26. Why are update operations failing?**

**Answer:**

Common causes include:

- Missing spintlyOrgUserId
- Invalid authentication
- Incorrect request payload
- Expired access token

---

**Q27. Why are duplicate users being created?****Answer:**

Duplicate users often result from creating users without checking for existing mappings or retrying failed operations without verifying whether the original request succeeded.

---

**18.14 Best Practices Questions****Q28. What is the recommended integration sequence?****Answer:**

1. Authenticate.
  2. Retrieve configuration.
  3. Cache configuration.
  4. Create user.
  5. Store Spintly identifiers.
  6. Assign permissions.
  7. Assign credentials.
  8. Validate access.
  9. Monitor synchronization.
- 

**Q29. What is the most common implementation mistake?****Answer:**

Failing to store spintlyOrgUserId immediately after user creation. Without this identifier, subsequent update, permission, and delete operations become unreliable.

---

**18.15 Quick Reference Table**

| Topic           | Recommendation                        |
|-----------------|---------------------------------------|
| Authentication  | Use OAuth and reuse valid tokens      |
| Configuration   | Cache Sites, Roles, and Access Points |
| Database        | Store all Spintly identifiers         |
| Synchronization | Track status and retry failures       |



| <b>Topic</b> | <b>Recommendation</b>                   |
|--------------|-----------------------------------------|
| Permissions  | Validate before assigning               |
| Credentials  | Verify assignment before testing        |
| Security     | Never expose secrets or tokens          |
| Logging      | Record all integration events           |
| Monitoring   | Review synchronization health regularly |

## **Chapter 19 – Glossary and Appendices**

---

### **19.1 Overview**

This chapter provides a consolidated reference for terminology, abbreviations, workflows, diagrams, and other supporting information used throughout this guide. It is intended to help readers quickly understand key concepts without searching through previous chapters.

---

### **19.2 Glossary**

| <b>Term</b>               | <b>Definition</b>                                                                                                 |
|---------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Access Point</b>       | A physical entry point, such as a door or gate, controlled by the Spintly platform.                               |
| <b>API</b>                | Application Programming Interface used for communication between the client application and the Spintly platform. |
| <b>Bearer Token</b>       | OAuth access token included in the Authorization header for protected API requests.                               |
| <b>Cache</b>              | Temporary local storage of configuration data such as Sites, Roles, and Access Points.                            |
| <b>Client Application</b> | The organization's software that integrates with the Spintly platform.                                            |
| <b>Client ID</b>          | Identifier issued by Spintly to authenticate a client application.                                                |
| <b>Client Secret</b>      | Confidential credential used with the Client ID during OAuth authentication.                                      |
| <b>Credential</b>         | A digital or physical identifier (such as a mobile credential or access card) used for authentication.            |
| <b>OAuth</b>              | Industry-standard authorization framework used for secure API authentication.                                     |
| <b>Reader</b>             | Spintly hardware device that validates credentials and controls access.                                           |
| <b>Role</b>               | A predefined set of permissions assigned to users.                                                                |

| Term                    | Definition                                                                         |
|-------------------------|------------------------------------------------------------------------------------|
| <b>Site</b>             | A physical location or organizational unit configured within the Spintly platform. |
| <b>Smart Access</b>     | The Spintly mobile application used to manage and present access credentials.      |
| <b>Synchronization</b>  | The process of keeping application data consistent with the Spintly platform.      |
| <b>spintlyOrgUserId</b> | Unique identifier assigned by Spintly to a user after successful creation.         |

---

### 19.3 Acronyms

#### Acronym Meaning

|       |                                    |
|-------|------------------------------------|
| API   | Application Programming Interface  |
| CRUD  | Create, Read, Update, Delete       |
| HTTP  | Hypertext Transfer Protocol        |
| HTTPS | Hypertext Transfer Protocol Secure |
| ID    | Identifier                         |
| JSON  | JavaScript Object Notation         |
| OAuth | Open Authorization                 |
| QA    | Quality Assurance                  |
| REST  | Representational State Transfer    |
| SSL   | Secure Sockets Layer               |
| TLS   | Transport Layer Security           |
| URI   | Uniform Resource Identifier        |
| URL   | Uniform Resource Locator           |

---

### 19.4 HTTP Status Code Reference

| Status Code     | Meaning                        | Typical Action             |
|-----------------|--------------------------------|----------------------------|
| 200 OK          | Request completed successfully | Continue processing        |
| 201 Created     | Resource created successfully  | Store returned identifiers |
| 400 Bad Request | Invalid request                | Validate payload           |

| Status Code               | Meaning                           | Typical Action                       |
|---------------------------|-----------------------------------|--------------------------------------|
| 401 Unauthorized          | Authentication failed             | Refresh OAuth token                  |
| 403 Forbidden             | Permission denied                 | Verify authorization                 |
| 404 Not Found             | Resource does not exist           | Check identifiers                    |
| 409 Conflict              | Duplicate or conflicting resource | Verify existing mappings             |
| 500 Internal Server Error | Server-side failure               | Retry if appropriate and review logs |

---

### 19.5 Recommended API Execution Sequence

Authenticate



Retrieve Configuration



Cache Configuration



Create User



Store spintlyOrgUserId



Assign Permissions



Assign Credential



Validate Access



Monitor Synchronization

---

## **19.6 User Lifecycle Summary**

Create User



Store Mapping



Assign Permissions



Assign Credential



User Active



Update User



Deactivate User



Delete User



Archive Mapping

---

## 19.7 Synchronization Workflow Summary

Business Event



API Request



Successful?

|

└— Yes

| Update Database

| Store Mapping

| Sync Complete

|

└─ No

↓

Log Error

↓

Retry

↓

Update Status

---

**19.8 Integration Architecture Summary**

Client Application

|

▼

Spintly Cloud APIs

|



Smart Access App

|



Spintly Reader

|



Physical Access

---

**19.9 Production Readiness Checklist**

Before deploying to production, ensure that:

| Task                               | Complete                 |
|------------------------------------|--------------------------|
| OAuth authentication implemented   | <input type="checkbox"/> |
| Client credentials stored securely | <input type="checkbox"/> |
| Configuration data cached          | <input type="checkbox"/> |
| User mappings implemented          | <input type="checkbox"/> |
| Synchronization validated          | <input type="checkbox"/> |
| Permissions assigned correctly     | <input type="checkbox"/> |
| Credentials verified               | <input type="checkbox"/> |
| Logging enabled                    | <input type="checkbox"/> |

| Task                       | Complete                 |
|----------------------------|--------------------------|
| Monitoring configured      | <input type="checkbox"/> |
| Error handling implemented | <input type="checkbox"/> |
| Retry strategy implemented | <input type="checkbox"/> |
| Documentation reviewed     | <input type="checkbox"/> |

---

## 19.10 References

This guide is based on the following implementation resources:

- **Spintly Type 1 Integration Guide**
- **Spintly Getting Started Guide**
- **Spintly Integration Examples**
- **Spintly Postman Collection / API Specification**
- **OAuth 2.0 Authorization Framework (RFC 6749)**

These resources should be consulted alongside this guide for the latest API endpoints, request formats, and platform-specific updates.

---

## 19.11 Document Revision History

| Version | Date   | Description                                                    |
|---------|--------|----------------------------------------------------------------|
| 1.0     |        | Initial Release Created comprehensive technical knowledge base |
| 1.1     | Future | API updates and documentation improvements                     |
| 1.2     | Future | Additional workflows and troubleshooting enhancements          |

---

## 19.12 Suggested Future Enhancements

As the integration evolves, consider extending this guide with:

- Complete API endpoint reference generated from the latest OpenAPI or Postman collection.
- Sequence diagrams for each major workflow.
- Sample implementations in multiple programming languages (e.g., Python, Java, C#, Node.js).
- CI/CD deployment recommendations.
- Monitoring dashboards and alerting examples.



- Advanced security topics such as secrets management and token rotation.
- Version migration guides for future API releases.